Clase 1 — Introducción al desarrollo de software y a la POO

La idea principal es:

> Programar no consiste solamente en escribir instrucciones que “funcionen”. Desarrollar software implica comprender un problema, diseñar una solución, implementarla, probarla y documentarla.

---

1. ¿Qué es el software?

En un sentido informal, hardware es la parte física y software es la parte lógica.

El software incluye:

- los programas y aplicaciones ejecutables;
- los archivos de configuración;
- la documentación;
- los modelos y diagramas UML;
- las pruebas.

Por ejemplo, si desarrollamos un sistema para supervisar sensores industriales, el software no es solamente el archivo ejecutable. *También forman parte del producto: el código fuente; las clases que representan los sensores; los archivos donde se almacenan las mediciones; el diagrama UML, etc.*

---

2. Programar no es lo mismo que desarrollar software

La programación es una parte del desarrollo de software. Podemos distinguir:

- **Programar:** escribir instrucciones en un lenguaje como C++ o Python.
- **Desarrollar software:** realizar todo el proceso necesario para obtener una solución útil, correcta y mantenible.

Un proceso de desarrollo típico comprende:

1. **Relevamiento:** comprender qué necesita el usuario.
2. **Análisis:** estudiar el problema, sus elementos y restricciones.
3. **Diseño:** decidir cómo estará organizada la solución.
4. **Implementación:** transformar el diseño en código.
5. **Pruebas:** comprobar que el programa funciona correctamente.
6. **Evaluación:** comparar el resultado con lo solicitado.

7. **Implantación:** poner la solución en funcionamiento.
8. **Mantenimiento:** corregirla, adaptarla o ampliarla posteriormente.

La diferencia puede verse con un ejemplo:

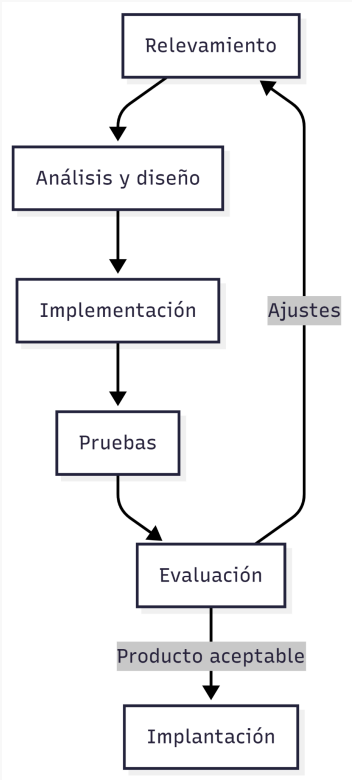
> “Necesito un programa que registre mediciones provenientes de un Arduino y después permita consultarlas”.

Escribir directamente una función que lea el puerto serie sería solamente programar. Antes conviene preguntarse: *¿Qué información contiene cada medición? ¿Cómo se identifica el dispositivo? ¿Dónde se almacenan los datos?...*

Esas preguntas pertenecen al análisis y al diseño.

---

3. El desarrollo es iterativo



Esto significa que normalmente no hacemos todo correctamente en un único intento.

Por ejemplo, en un primer intento podríamos construir solamente una clase `Sensor`. Luego, al probarla, advertir que necesitamos diferenciar sensores de temperatura y presión. Entonces revisamos el modelo y hacemos una nueva iteración.

El **diseño** no es algo que se hace una vez y queda congelado. Se mejora a medida que comprendemos mejor el problema.

---

4. ¿Qué se necesita para desarrollar software?

Herramientas

Compiladores, editores, gestores de proyectos, herramientas de modelado.

Conocimientos teóricos

Por ejemplo: orientación a objetos; estructuras de datos; UML.

Habilidades técnicas

Saber escribir, compilar, probar y depurar un programa.

Habilidades blandas

También son necesarias: interpretar lo que solicita otra persona; explicar una decisión técnica; documentar; trabajar en equipo.

Requerimiento específico

Debe existir una necesidad que el software intente resolver. Si el requerimiento está mal entendido, podemos construir un programa que no resuelva el problema real.

Proceso de desarrollo

Las actividades deben realizarse con cierto orden y criterio. La herramienta por sí sola no garantiza una buena solución.

---

5. Tecnologías que pueden intervenir

Un sistema real generalmente combina diferentes tecnologías:

- **Programación:** C++, Python, JavaScript, etc.
- **Comunicación:** redes, puertos serie, protocolos.
- **Almacenamiento:** archivos y bases de datos.
- **Documentación:** diagramas, manuales y bases de conocimiento.
- **Inteligencia artificial:** asistentes y herramientas de apoyo.

Esta materia utilizará principalmente:

- C++ y Python para programar;
- UML para modelar;
- herramientas de red para construir aplicaciones cliente-servidor;
- archivos y posiblemente bases de datos para conservar información.

La POO no trabaja aislada: ayuda a organizar todos esos componentes.

---

6. Herramientas: editor, compilador, IDE y CASE

Editor de código

Permite escribir y modificar archivos fuente.

En nuestro entorno, **VS Code** cumple principalmente esta función, aunque mediante extensiones también integra otras herramientas.

Compilador

Traduce el código C++ a lenguaje ejecutable.

En Linux utilizamos:

```
g++
```

Por ejemplo:

```
g++ main.cpp Sensor.cpp -o programa
```

Herramienta de construcción

Automatiza la compilación, especialmente cuando hay muchos archivos.

En la materia utilizamos **`make`** y un **`Makefile`**.

Depurador

Permite detener el programa, avanzar instrucción por instrucción e inspeccionar variables. Una herramienta habitual es `gdb`.

IDE

Un entorno de desarrollo integrado reúne varias de las funciones anteriores: *edición; compilación; depuración; gestión del proyecto; visualización de errores; integración de extensiones*.

Es importante entender que VS Code no reemplaza al compilador: normalmente ejecuta por detrás a `g++`, `make` o `gdb`.

Herramienta CASE

CASE significa **Computer-Aided Software Engineering**. Son herramientas destinadas a asistir el análisis, diseño y documentación del software.

En la materia usamos **Umbrello**, principalmente para:

- crear diagramas UML;
- representar clases y relaciones;

Ninguna herramienta piensa el diseño por nosotros. Puede generar código a partir de un diagrama, pero si el modelo es incorrecto, generará una estructura incorrecta.

---

7. ¿Qué es un paradigma de programación?

Un paradigma es una forma general de pensar, organizar y expresar una solución.

No es solamente una característica de sintaxis. Cambia la manera en que descomponemos un problema.

Programación imperativa

Le indicamos a la computadora qué acciones debe ejecutar y en qué orden.

Por ejemplo:

```
leerTemperatura();
calcularPromedio();
mostrarResultado();
```

Dentro de la programación imperativa aparecen:

- **programación estructurada**;
- programación modular;
- programación orientada a objetos.

Programación declarativa: NO USAMOS

Describimos qué resultado o relación queremos, sin detallar necesariamente toda la secuencia de ejecución. Aquí aparecen paradigmas como:

- programación funcional;
- programación lógica;
- programación con restricciones.

La POO no reemplaza completamente a la **programación estructurada**. Un método de una clase sigue usando:

- secuencias;
- decisiones con `if`;
- repeticiones con `for` o `while`;
- funciones.

Los paradigmas se pueden combinar. C++ y Python son lenguajes **multiparadigma**.

8. De la programación estructurada a la orientación a objetos

En programación estructurada, solemos organizar el programa alrededor de funciones.

Por ejemplo:

```
double temperatura;

void actualizarTemperatura(double valor)
{
    temperatura = valor;
}

void mostrarTemperatura()
{
    cout << temperatura << endl;
}
```

El **centro** de la solución son las **operaciones**. El dato `temperatura` aparece separado y podría ser global.

En orientación a objetos, agrupamos los estados y las operaciones relacionadas:

```
class SensorTemperatura
{
private:
    double temperatura;

public:
    void actualizar(double valor)
    {
        temperatura = valor;
    }

    void mostrar()
    {
        cout << temperatura << endl;
    }
};
```

Ahora existe una entidad `SensorTemperatura` que reúne:

- **estado:** la temperatura almacenada;
- **comportamiento:** actualizarla y mostrarla.

La diferencia importante no es simplemente haber agregado la palabra `class`. Cambió la forma de organizar la solución.

---

9. Objeto y clase: primera aproximación

Clase

Es una definición o modelo que indica:

- qué datos tendrán los objetos;
- qué operaciones podrán realizar.

Por ejemplo:

```
class SensorTemperatura;
```

representa un tipo de sensor.

Objeto

Es una instancia concreta de una clase.

```
SensorTemperatura sensorInterior;
SensorTemperatura sensorExterior;
```

Tenemos dos objetos diferentes construidos a partir de la misma clase. Cada uno conserva su propio estado.

---

10. Los 4 pilares de la POO

Abstracción

Consiste en representar solamente las características relevantes para el problema.

Un sensor real tiene materiales, dimensiones, componentes electrónicos y muchas otras propiedades. Pero en un programa quizá sólo necesitemos:

- identificador;
- tipo;
- operación para obtener una medición.

Abstraer es decidir qué incluir en el modelo y qué dejar afuera.

Encapsulamiento

Consiste en reunir los datos y operaciones correspondientes, **controlando cómo se accede al estado interno**.

Por eso normalmente los atributos se declaran `private` y se ofrecen operaciones públicas controladas.

Herencia

Permite definir una nueva clase utilizando y especializando otra clase existente.

Polimorfismo

Permite tratar objetos diferentes mediante una interfaz común, dejando que cada uno responda de acuerdo con su tipo concreto.

Por ejemplo, podríamos solicitar `leer()` a distintos sensores, aunque cada sensor implemente internamente esa lectura de una manera diferente.

---

11. Descomposición funcional frente a descomposición orientada a objetos

En un enfoque orientado a objetos, primero preguntamos:

> ¿Qué entidades participan y qué responsabilidades tiene cada una?

Podrían aparecer:

```
Sensor
Medicion
ArchivoMediciones
```

Luego asignamos **comportamientos**:

- `Sensor` obtiene una medición.
- `Medicion` representa los datos capturados.
- `ArchivoMediciones` guarda y recupera mediciones.

No significa que en POO desaparezcan las funciones. Las funciones pasan a formar parte de clases y se denominan normalmente **métodos**.

---

12. Ventajas de la POO

Cercanía con el dominio del problema. Podemos construir modelos usando conceptos del problema real: sensores, clientes, archivos, pedidos, motores o alarmas.

Modularidad. El sistema se divide en unidades con responsabilidades concretas.

Encapsulamiento. Cada objeto controla su propio estado y no permite el acceso de cualquiera a sus datos.

Reutilización. Una clase bien diseñada puede utilizarse en diferentes partes del programa o en futuros proyectos.

Trabajo en equipo. Diferentes integrantes pueden trabajar sobre clases o subsistemas distintos, siempre que las interfaces estén definidas claramente.

Sin embargo: Usar clases no garantiza automáticamente estas ventajas. Podemos escribir código orientado a objetos muy desordenado si creamos clases sin responsabilidades claras o con dependencias innecesarias.

---

13. Desventajas y costos

La POO también introduce dificultades:

- requiere más esfuerzo inicial de análisis;
- puede necesitar más archivos y más código;
- tiene una curva de aprendizaje;
- no todos los problemas necesitan una solución orientada a objetos.

Para un programa de diez líneas, crear una gran jerarquía de clases sería innecesario. La orientación a objetos resulta especialmente útil cuando el programa:

- tiene cierta complejidad;
- contiene varias entidades relacionadas;
- es desarrollado por varias personas.

---

14. Análisis, diseño e implementación orientados a objetos

Análisis orientado a objetos

Busca comprender el problema. Preguntas típicas:

- ¿Qué necesita el usuario?
- ¿Qué información debemos representar?
- ¿Qué responsabilidades existen?

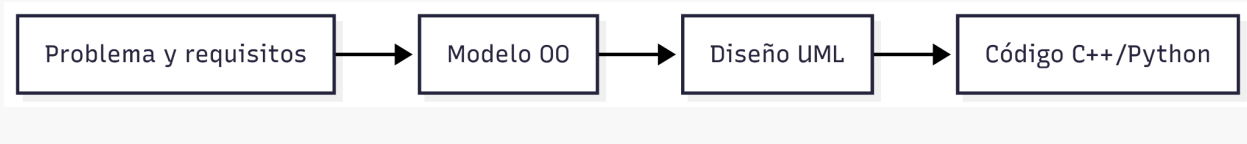
Diseño orientado a objetos

Transforma la comprensión del problema en una estructura de software. Aquí decidimos:

- qué clases existirán;
- qué atributos tendrán;
- qué operaciones ofrecerán;
- cómo se relacionarán;
- qué clase será responsable de cada tarea.

Implementación orientada a objetos

El diseño se convierte en código C++ o Python.



Uno de los mensajes más importantes de la clase es que un desarrollo orientado a objetos no consiste únicamente en programar clases:

> Requiere **análisis** OO, **diseño** OO e **implementación** en un lenguaje que permita expresar ese diseño.

---

15. UML y modelado visual

UML significa **Unified Modeling Language**, lenguaje unificado de modelado.

Es una notación gráfica utilizada para representar sistemas de software. No es un lenguaje de programación y los diagramas no se ejecutan.

Un diagrama de clases puede mostrar:

- clases;
- atributos;
- operaciones;
- relaciones;
- multiplicidades;
- herencia.

UML sirve para:

- pensar la estructura antes de implementarla;
- detectar problemas;
- documentar;
- facilitar cambios;
- generar una base de código con herramientas CASE.

La intención es que el modelo ayude a construir la solución.

Lo esencial de la clase

Si tuvieras que quedarte con cinco ideas, serían estas:

1. **El software no es solamente código:** también incluye datos, modelos, pruebas y documentación.
2. **Programar es sólo una etapa del desarrollo:** antes hay que comprender y diseñar; después hay que probar, evaluar y mantener.
3. **La POO es una forma de organizar una solución alrededor de objetos y clases**, combinando estado y comportamiento.
4. **UML permite modelar la solución antes y durante la implementación.** Umbrello ayuda a trabajar con esos modelos, pero no reemplaza las decisiones de diseño.
5. **C++ es la herramienta de implementación, no el objetivo final.** El objetivo es aprender a construir software orientado a objetos correctamente.

Para comprobar que quedó clara la base, tratá de responder:

- *¿Por qué un programa ejecutable no constituye por sí solo todo el software?*
- *¿Cuál es la diferencia entre análisis, diseño e implementación?*
- *¿Qué diferencia fundamental hay entre organizar un programa alrededor de funciones y organizarlo alrededor de objetos?*
- *¿Por qué generar código desde Umbrello no garantiza que el diseño sea correcto?*

Clase 2 — De C a objetos y clases en C++

1. C++ como extensión de C

C++ nació como una extensión de C. La diferencia es que C++ agrega, entre otras cosas:

- referencias;
- parámetros predeterminados;
- sobrecarga;
- clases y objetos;
- constructores;
- control de visibilidad;
- herencia;
- polimorfismo;

Parte A — Cambios básicos de C a C++

2. Archivos de cabecera

En C era habitual escribir:

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
```

En C++, las cabeceras equivalentes suelen escribirse sin `.h` y con una `c` inicial:

```
#include <cstdio>
#include <cstdlib>
#include <cmath>
```

Las bibliotecas propias de C++ se incluyen así:

```
#include <iostream>
#include <string>
```

Por ejemplo, `iostream` contiene los elementos utilizados para entrada y salida:

- `std::cin`;
- `std::cout`;
- `std::endl`.

---

3. Espacios de nombres y operador `::`

Un **namespace** es una región que agrupa identificadores y evita conflictos de nombres.

`cout` pertenece al espacio de nombres `std`:

```
std::cout << "Hola" << std::endl;
```

El operador `::` se denomina **operador de resolución de ámbito**. Indica dónde está definido un nombre.

```
std::cout
 ↑     ↑
ámbito elemento
```

Alternativa:

```
using namespace std;

cout << "Hola" << endl;
```

Al escribir `using namespace std`, permitimos usar directamente los nombres contenidos en `std`.

---

4. Declaración simplificada de estructuras y enumeraciones

En C, una estructura suele declararse de esta manera:

```
struct Material
{
    long codigo;
    char nombre[20];
};

struct Material soporte;
```

En C++ no es necesario repetir `struct` al declarar una variable:

```
struct Material
{
    long codigo;
    char nombre[20];
};

Material soporte;
```

Lo mismo sucede con una enumeración:

```
enum Color
{
    ROJO,
    AMARILLO,
    VERDE
};

Color frente;
```

Una enumeración permite definir un conjunto limitado de valores con **nombres significativos**. Es más claro escribir:

```
Direccion direccion = DERECHA;
```

que utilizar un número cuyo significado no resulta evidente:

```
int direccion = 2;
```

---

5. Entrada y salida con `cin` y `cout`

En C se suelen utilizar `scanf()` y `printf()`:

```c
scanf("%f", &base);
printf("Area: %f", area);
```

En C++ utilizamos flujos:

```cpp
std::cin >> base;
std::cout << "Area: " << area << std::endl;
```

Los operadores tienen los siguientes sentidos:

```
cin  >> variable
     extrae un dato del flujo de entrada

cout << dato
     inserta un dato en el flujo de salida
```

Una ventaja es que no debemos indicar manualmente el formato con `%d`, `%f`, etc. **El operador `<<` reconoce el tipo del dato**.

---

6. Conversión de tipos o casting

```cpp
int x = 10;
int y = 3;
```

Si hacemos:

```cpp
std::cout << x / y;
```

el resultado es:

```
3
```

Como ambos operandos son enteros, C++ realiza división entera. Para obtener un resultado real, al menos uno debe convertirse:

```cpp
std::cout << static_cast<float>(x) / y;
```

Resultado:

```
3.33333
```

Tambien se puede realizar una conversión del estilo:

```cpp
float(x)
```

También existe el estilo heredado de C:

```cpp
(float)x
```

---

7. Parámetros con valores predeterminados

C++ permite asignar valores por defecto a los parámetros de una función.

```
double velocidadFinal(double altura,
                      double velocidadInicial = 0.0,
                      double gravedad = 9.8);
```

Podemos llamar a la misma función de varias maneras:

```
velocidadFinal(100);
```

Equivale a:

```
velocidadFinal(100, 0.0, 9.8);
```

También podemos reemplazar algunos valores:

```
velocidadFinal(100, 10);
```

Aquí se utiliza:

```
altura = 100; velocidadInicial = 10; gravedad = 9.8
```

O indicar todos:

```
velocidadFinal(100, 10, 1.6);
```

Los parámetros con valores predeterminados deben quedar a la derecha:

```
// Correcto
void funcion(int a, int b = 2, int c = 3);

// Incorrecto
void funcion(int a = 1, int b, int c);
```

8. Paso por valor, puntero y referencia

Paso por valor

```
void duplicar(int numero)
{
    numero = numero * 2;
}
```

Si hacemos:

```
int x = 5;
duplicar(x);
```

`x` continúa valiendo `5`, porque la función recibió una copia.

Paso mediante puntero

En **C** podríamos modificar la variable original pasando su dirección:

```
void duplicar(int* numero)
{
    *numero = *numero * 2;
}

int main()
{
    int x = 5;
    duplicar(&x);
}
```

Aquí:

- `&x` obtiene la dirección de `x`; `numero` guarda esa dirección; `*numero` accede al contenido ubicado allí.

Paso mediante referencia

C++ incorpora referencias:

```
void duplicar(int& numero)
{
    numero = numero * 2;
}

int main()
{
    int x = 5;
    duplicar(x);
}
```

`numero` es un alias de `x`. No es una copia: ambos nombres se refieren al mismo dato.

Por eso el cambio realizado dentro de la función **modifica a `x`**.

---

9. Reserva dinámica: `new` y `delete`

C++ permite crear datos dinámicamente:

```
int* numero = new int;
```

`new`:

1. reserva memoria;
2. crea el dato;

3. devuelve un puntero hacia ese dato.

Para liberar la memoria:

```
delete numero;
```

Con un arreglo:

```
char* texto = new char[50];

// Uso del arreglo...

delete[] texto;
```

Si no liberamos la memoria, aparece una **fuga de memoria**: el programa pierde acceso a una región que sigue reservada.

Sin embargo, no significa que todos los objetos deban crearse con `new`. En C++ moderno, cuando no necesitamos vida dinámica, normalmente **preferimos objetos directos**.

Parte B — De la programación modular a la POO

10. Programación modular

La **programación modular** intenta reducir la complejidad dividiendo el programa en componentes. Cada módulo reúne elementos relacionados con una responsabilidad.

Por ejemplo:

```
Programa
├── módulo de archivos
├── módulo de cálculos
├── módulo de comunicación
└── módulo de interfaz
```

Un módulo no suele ser una única función. Un módulo puede contener:

- constantes;
- varias funciones relacionadas;
- una o varias clases.

En C++, normalmente se representa mediante **archivos** `.h` y `.cpp`.

---

11. Programas multiarchivo y compilación separada

Supongamos este proyecto:

```
main.cpp
AutoEstereo.h
AutoEstereo.cpp
```

Cada archivo `.cpp` puede compilarse de manera independiente:

```
g++ -c main.cpp
g++ -c AutoEstereo.cpp
```

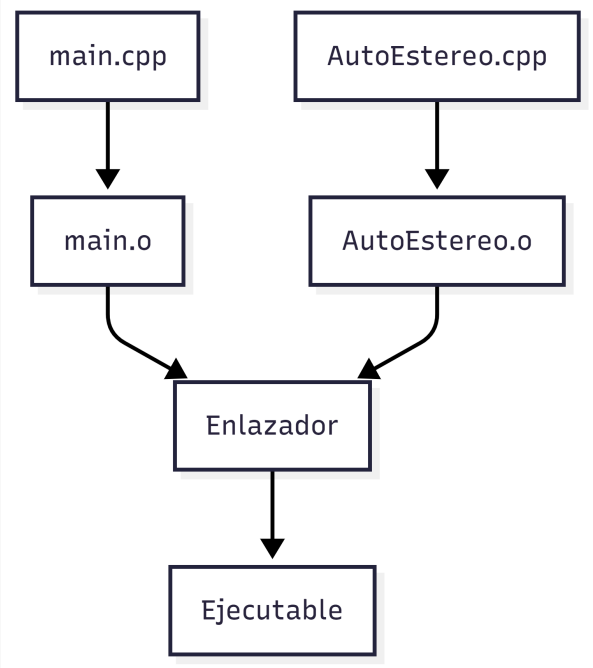
Se obtienen archivos objeto:

```
main.o
AutoEstereo.o
```

Después se enlazan:

```
g++ main.o AutoEstereo.o -o programa
```

El proceso completo es:



Esto explica lo que hace el `Makefile`. No crea mágicamente el programa: **automatiza la compilación y el enlazado**.

---

12. ¿Por qué la programación modular no alcanzaba?

Dividir el programa en archivos ayuda, pero no resuelve por sí solo todos los problemas. En sistemas grandes pueden aparecer:

- datos globales accedidos desde muchos módulos;
- funciones de distintos archivos modificando los mismos datos;
- cambios pequeños que afectan numerosas partes.

Aunque repartamos las variables y funciones en módulos, siguen siendo **elementos separados**. Nada obliga a que el estado del estéreo sea manipulado solamente por las funciones correctas. La **POO propone reunirlos** dentro de una **CLASE**:

```
class AutoEstereo
{
private:
    bool encendido;
    int volumen;
    float frecuencia;
public:
    void encender();
    void subirVolumen();
    void cambiarFrecuencia(float nuevaFrecuencia);
};
```

La clase agrupa:

- el **estado** del autoestéreo;
- las **operaciones** permitidas sobre ese estado.

Parte C — Modelo de objetos

13. Fenómeno y concepto

Fenómeno

Es un **elemento concreto** que identificamos en el dominio:

- este autoestéreo Alpine;
- aquel teléfono particular;
- un libro específico;

Concepto

Es una **abstracción** que agrupa características comunes:

- autoestéreo;

- teléfono;
- libro;

El concepto tiene:

- **nombre:** identifica la categoría;
- **propósito:** indica qué propiedades debe cumplir algo para pertenecer;
- **miembros:** elementos concretos pertenecientes al concepto.

En programación, esta idea se aproxima a:

```
Concepto  → Clase
Fenómeno  → Objeto
```

---

14. Clase y objeto

Clase

Una clase define las características compartidas por un conjunto de objetos

- atributos;
- operaciones;
- reglas de acceso.

Objeto

Un objeto es una instancia concreta de esa clase.

```
AutoEstereo estereoPedro;
AutoEstereo estereoFacultad;
```

Son **dos objetos diferentes**. Comparten la misma definición, pero cada uno mantiene su propio estado. Por ejemplo:

```
estereoPedro
    encendido = true
    volumen = 20

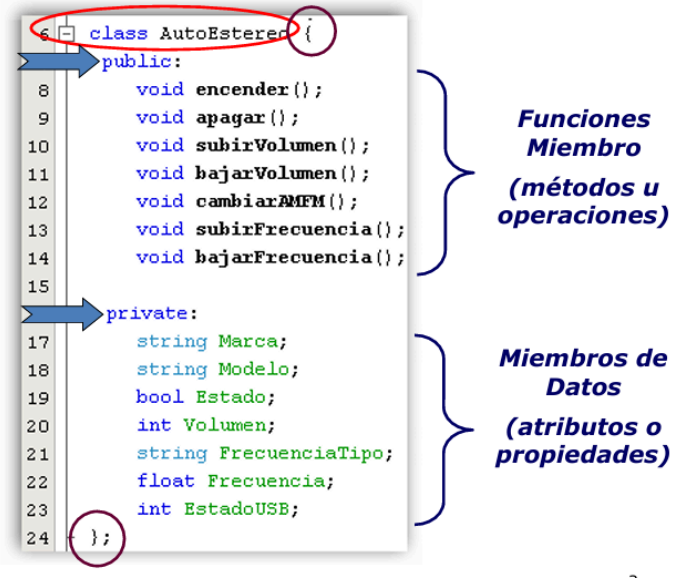
estereoFacultad
    encendido = false
    volumen = 5
```

La clase se escribe una vez, pero permite crear muchos objetos.

---

15. Atributos y comportamiento

Para modelar un autoestéreo, identificamos:



Atributos

Describen su estado:

- marca;
- modelo;
- encendido o apagado;
- volumen;

Comportamientos

Representan aquello que puede hacer:

- encenderse;
- apagarse;
- subir o bajar el volumen;

La pregunta central al analizar es: **¿Qué necesita saber y qué necesita hacer?**

En C++:

```
Lo que sabe → atributos o miembros de datos
Lo que hace → métodos o funciones miembro
```

---

16. Abstracción

Abstraer significa elegir qué características son importantes desde un enfoque particular. Un autoestéreo real tiene:

- componentes electrónicos;
- peso;
- dimensiones;

Pero si queremos controlar la radio, posiblemente sólo necesitemos:

```
estado
volumen
frecuencia
```

No estamos representando toda la realidad. Construimos un **modelo adecuado** para nuestro problema.

---

17. Encapsulamiento e interfaz

Encapsular significa mantener juntos los datos y las operaciones correspondientes, ocultando los detalles internos. Podemos imaginar el objeto como una cápsula:

```
Exterior: interfaz pública
    encender()
    apagar()

Interior: representación privada
    estado
```

El usuario del objeto sabe **qué** puede pedirle, pero no necesita conocer exactamente **cómo** lo hace. Por ejemplo:

```
estereo.encender();
```

Desde afuera solamente solicitamos que se encienda. No necesitamos saber qué variable interna cambia. **Esto permite modificar la implementación sin alterar el código que utiliza la clase**.

---

18. Modularidad, jerarquía y comunicación

El modelo orientado a objetos incluye cinco ideas:

- abstracción;
- encapsulamiento;
- modularidad;
- jerarquía;
- comunicación.

Modularidad

Las clases se organizan en componentes y archivos manejables.

Jerarquía

Las abstracciones pueden clasificarse de lo general a lo particular:

```
Dispositivo
└── Reproductor
    └── AutoEstereo
```

Esto prepara el terreno para estudiar herencia más adelante.

Comunicación

Los **objetos colaboran mediante mensajes**. En C++, un mensaje se materializa generalmente como una llamada a un método:

```
estereo.encender();
```

Podemos leerlo como: ell código cliente envía al objeto `estereo` el mensaje `encender`.

El receptor del mensaje es `estereo` y la operación solicitada es `encender()`.

Parte D — Implementación de clases en C++

19. Estructura de una clase

La clase `AutoEstereo` podría declararse así:

```
class AutoEstereo
{
public:
    void encender();
    void apagar();
    void subirVolumen();
    void bajarVolumen();
private:
    std::string marca;
    std::string modelo;
    bool encendido;
    int volumen;
    float frecuencia;
};
```

---

20. Visibilidad: `public`, `private` y `protected`

`public`

Los elementos públicos pueden utilizarse desde afuera:

```
estereo.encender();
```

`private`

Los elementos privados sólo pueden ser accedidos directamente por funciones de la propia clase:

```
estereo.encendido = true; // Error si encendido es private
```

La modificación debe realizarse mediante la interfaz:

```
estereo.encender();
```

`protected`

Es parecido a `private`, pero también permite acceso desde clases derivadas. Será importante al estudiar herencia.

`friend` permite otorgar acceso especial a otra función o clase, pero rompe parcialmente la barrera de encapsulamiento y debe utilizarse con cuidado.

---

21. Separación entre `.h` y `.cpp`

La cátedra trabaja con dos archivos por clase.

`AutoEstereo.h`

Contiene principalmente la declaración:

```
#ifndef AUTOESTEREO_H
#define AUTOESTEREO_H
#include <string>

class AutoEstereo
{
public:
    void encender();
    void apagar();
private:
    bool encendido;
};

#endif
```

Este archivo indica **qué contiene y qué ofrece** la clase.

`AutoEstereo.cpp`

Contiene la implementación:

```
#include "AutoEstereo.h"

void AutoEstereo::encender()
{
    encendido = true;
}

void AutoEstereo::apagar()
{
```

```
    encendido = false;
}
```

En:

```
void AutoEstereo::encender()
```

tenemos:

```
void           → tipo de retorno
AutoEstereo::  → clase a la que pertenece
encender       → nombre del método
```

main.cpp

El programa cliente sólo necesita **incluir el encabezado**:

```
#include "AutoEstereo.h"

int main()
{
    AutoEstereo estereo;

    estereo.encender();
    estereo.apagar();

    return 0;
}
```

---

22. Creación directa y dinámica

Objeto directo

```
AutoEstereo estereo;
estereo.encender();
```

Para acceder a sus miembros utilizamos el punto:

```
objeto.metodo();
```

El objeto se destruye automáticamente cuando termina su ámbito.

Objeto dinámico

```
AutoEstereo* estereo = new AutoEstereo;

estereo->encender();
estereo->apagar();

delete estereo;
```

Como `estereo` es un puntero, utilizamos:

```
puntero->metodo();
```

La diferencia esencial es:

Declaración	Acceso	Liberación
`AutoEstereo estereo;`	`estereo.encender()`	Automática
`AutoEstereo* estereo = new AutoEstereo;`	`estereo->encender()`	`delete estereo`

---

23. Métodos `get` y `set`

Como los atributos son privados, a veces necesitamos operaciones públicas para consultarlos o modificarlos.

Getter

Devuelve el valor:

```
bool AutoEstereo::getEstado() const
{
    return encendido;
}
```

Uso:

```
std::cout << estereo.getEstado();
```

Setter

Recibe y asigna un valor:

```
void AutoEstereo::setFrecuencia(float frecuencia)
{
    this->frecuencia = frecuencia;
}
```

`this` es un puntero al objeto que está ejecutando el método. Tenemos:

```
this->frecuencia → atributo del objeto
frecuencia       → parámetro recibido
```

No es obligatorio escribir un getter y setter para cada atributo.

---

24. Constructor

El constructor es una función que se ejecuta automáticamente cuando se crea un objeto. Características:

- tiene el mismo nombre que la clase;
- no tiene tipo de retorno;
- inicializa el objeto;
- puede recibir parámetros.

Declaración:

```
class AutoEstereo
{
public:
    AutoEstereo(std::string marca, std::string modelo);

private:
    std::string marca;
    std::string modelo;
};
```

```
AutoEstereo::AutoEstereo(std::string marca, std::string modelo)
{
    this->marca = marca;
    this->modelo = modelo;
}
```

Creación:

```
AutoEstereo estereo("Alpine", "iXA-W404");
```

El constructor le da un estado inicial válido. Por ejemplo, también debería definir:

```
encendido = false
volumen = 0
frecuencia = 0
```

Si un atributo queda sin inicializar, puede contener un valor indeterminado.

---

25. Sobrecarga

La sobrecarga permite definir varias funciones con el mismo nombre, siempre que tengan diferentes listas de parámetros.

```
void setFrecuencia(float frecuencia);
void setFrecuencia(float frecuencia, std::string banda);
```

El compilador decide cuál ejecutar según los argumentos:

```
estereo.setFrecuencia(99.9f);
estereo.setFrecuencia(103.1f, "FM");
```

---

26. Constructores sobrecargados

Una clase puede ofrecer diferentes maneras de crear objetos:

```
class AutoEstereo
{
public:
    AutoEstereo();
    AutoEstereo(std::string marca, std::string modelo);
    AutoEstereo(std::string banda, float frecuencia);
};
```

Entonces podemos escribir:

```
AutoEstereo generico;
AutoEstereo alpine("Alpine", "iXA-W404");
AutoEstereo radio("FM", 99.9f);
```

Ejemplo integrador de la clase

La relación completa puede verse así:

AutoEstereo.h

```
#ifndef AUTOESTEREO_H
#define AUTOESTEREO_H
#include <string>

class AutoEstereo
{
public:
    AutoEstereo(std::string marca, std::string modelo);

    void encender();
    void apagar();
    void subirVolumen();
    bool getEstado() const;
    int getVolumen() const;
private:
    std::string marca;
    std::string modelo;
    bool encendido;
```

```
    int volumen;
};

#endif
```

AutoEstereo.cpp

```
#include "AutoEstereo.h"

AutoEstereo::AutoEstereo(std::string marca, std::string modelo)
{
    this->marca = marca;
    this->modelo = modelo;
    encendido = false;
    volumen = 0;
}

void AutoEstereo::encender()
{
    encendido = true;
}

void AutoEstereo::apagar()
{
    encendido = false;
}

void AutoEstereo::subirVolumen()
{
    if (encendido && volumen < 100)
    {
        volumen++;
    }
}

bool AutoEstereo::getEstado() const
{
    return encendido;
}

int AutoEstereo::getVolumen() const
{
    return volumen;
}
```

main.cpp

```cpp
#include <iostream>
#include "AutoEstereo.h"

int main()
{
    AutoEstereo estereo("Alpine", "iXA-W404");
    estereo.encender();
    estereo.subirVolumen();

    std::cout << "Volumen: " << estereo.getVolumen() << std::endl;
    return 0;
}
```

Compilación:

```
g++ main.cpp AutoEstereo.cpp -o autoestereo
```

Ejecución:

```
./autoestereo
```

Lo esencial de la segunda clase

1. C++ conserva las estructuras básicas de C, pero añade herramientas.
2. Una referencia es un alias de otra variable y permite modificarla.
3. Un módulo no es simplemente una función: es una unidad funcional que puede estar formada por varios elementos y archivos.
4. En la compilación separada, cada `.cpp` se compila y luego los archivos objeto se enlazan para generar el ejecutable.
5. Una clase define atributos y comportamientos compartidos; un objeto es una instancia concreta.
6. El encapsulamiento separa la interfaz pública de la representación privada.
7. En C++, la declaración de una clase suele colocarse en `.h` y la implementación de sus métodos en `.cpp`.
8. Los objetos directos usan `.` y los punteros a objetos usan `->`.
9. El constructor inicializa el objeto cuando se crea.
10. La sobrecarga permite repetir un nombre con diferentes listas de parámetros.

TP1 — Actividades 2, 3 y 4

El TP1 completo sigue una secuencia muy coherente:

Actividad	Pregunta que responde	Resultado
2	¿Cómo se construye automáticamente un programa con varios archivos?	`Makefile`
3	¿Cómo se diseña una clase antes de programarla?	Diagrama UML y código base generado por Umbrello
4	¿Cómo se completa, compila y prueba esa clase?	Programa funcional de simulación de móviles

La idea global fue pasar de la construcción de programas multiarchivo al desarrollo orientado a objetos:

```
Compilación manual
      ↓
Automatización con make
      ↓
Modelado UML de la clase Movil
      ↓
Generación de Movil.h y Movil.cpp
      ↓
Implementación del comportamiento
      ↓
Pruebas desde main.cpp
```

Actividad 2 — `make` y `Makefile`

1. ¿Qué se buscaba aprender?

Su propósito era comprender cómo se compila un programa formado por varios archivos.

En un programa muy pequeño podemos escribir manualmente:

```
g++ hola.cpp -o saludo
```

Pero si el proyecto contiene:

```
main.cpp
Sensor.cpp
Sensor.h
```

```
Alarma.cpp
Alarma.h
```

el comando empieza a crecer:

```
g++ main.cpp Sensor.cpp Alarma.cpp -o sensores
```

En proyectos mayores, escribir siempre el comando completo es incómodo y propenso a errores. Para automatizarlo se utiliza:

```
make + Makefile
```

- `make` es el programa que interpreta las reglas.
- `Makefile` es el archivo donde escribimos esas reglas.

---

2. Primera parte: compilación manual

La consigna pedía compilar `hola.cpp` mediante:

```
g++ hola.cpp -o saludo
```

Cada parte significa:

```
g++         compilador de C++
hola.cpp    archivo fuente
-o saludo   nombre del ejecutable generado
```

El resultado era un archivo ejecutable llamado:

```
saludo
```

Luego se verificaba mediante:

```
./saludo
```

El prefijo `./` significa que el ejecutable se encuentra en el directorio actual.

Después se utilizaba:

```
ls -la
```

para observar los archivos presentes. Esto permitía comprobar que, a partir del código fuente, apareció un nuevo archivo ejecutable.

---

3. ¿Qué ocurría con `make hola` sin un Makefile?

La consigna pedía probar:

```
make hola
```

Aunque no hubiera un `Makefile`, `make` conoce algunas **reglas implícitas**.

Si encuentra:

```
hola.cpp
```

puede inferir aproximadamente esta regla:

```
hola.cpp → compilar → hola
```

Por lo tanto, ejecuta un comando equivalente a:

```
g++ hola.cpp -o hola
```

La diferencia respecto de la compilación anterior es el nombre:

```
g++ hola.cpp -o saludo  → genera saludo
make hola               → genera hola
```

Esto servía para comprobar que `make` no necesita obligatoriamente un `Makefile` para casos muy simples: conoce algunas reglas básicas asociadas a extensiones conocidas.

---

4. ¿Qué ocurría con `make saludo`?

Luego se pedía probar:

```
make saludo
```

Sin un `Makefile`, `make` busca alguna forma de construir un archivo llamado `saludo`.

Por ejemplo, intentaría encontrar algo como:

```
saludo.cpp
```

Pero el archivo existente era:

```
hola.cpp
```

`make` no puede adivinar que queremos compilar `hola.cpp` y llamar `saludo` al resultado. Para establecer esa relación necesitamos una regla explícita.

---

5. Estructura de una regla

La consigna proponía:

```
main: hola.cpp
    g++ -o saludo hola.cpp
```

```
clean:
    rm -f saludo
```

Una regla tiene esta estructura:

```
objetivo: dependencias
    receta
```

En este caso:

```
objetivo     → main
dependencia  → hola.cpp
receta       → g++ -o saludo hola.cpp
```

Cuando escribimos:

```
make main
```

`make` busca la regla cuyo objetivo se llama `main` y ejecuta su receta.

¿Por qué el objetivo se llama `main` pero genera `saludo`?

La regla de la consigna permite ver que el nombre del objetivo es una etiqueta que `make` utiliza para encontrar la regla.

Sin embargo, no es el mejor diseño, porque la receta no genera un archivo llamado `main`, sino uno llamado `saludo`. Como `main` no aparece nunca, `make main` probablemente volverá a compilar cada vez.

Una correspondencia más natural sería:

```
saludo: hola.cpp
    g++ -o saludo hola.cpp
```

Ahora el objetivo coincide con el archivo generado.

---

6. Regla `clean`

La regla:

```
clean:
    rm -f saludo
```

se ejecuta mediante:

```
make clean
```

`rm` elimina archivos y la opción `-f` evita que aparezca un error si el archivo no existe.

En tu captura primero apareció correctamente:

```
make clean
```

y se eliminó el ejecutable.

Esto muestra que `clean` no es una palabra mágica de `make`. Funciona porque nosotros definimos una regla con ese nombre.

---

7. Programa multiarchivo `sensores`

La segunda parte utilizó:

```
main.cpp
Sensor.cpp
Sensor.h
Alarma.cpp
Alarma.h
```

El `Makefile` que incluiste fue:

```
sensores: main.cpp Alarma.cpp Sensor.cpp
    g++ -o sensores main.cpp Alarma.cpp Sensor.cpp

clean:
    rm -f sensores
```

Regla `sensores`

```
sensores: main.cpp Alarma.cpp Sensor.cpp
```

Significa:

> Para construir `sensores`, necesito `main.cpp`, `Alarma.cpp` y `Sensor.cpp`.

La receta:

```
g++ -o sensores main.cpp Alarma.cpp Sensor.cpp
```

compila los tres archivos y los enlaza para producir un único ejecutable.

Después se utiliza:

```
make sensores
```

o simplemente:

```
make
```

Como `sensores` es la primera regla, se convierte en el objetivo predeterminado.

---

8. ¿Cómo decide `make` si debe recompilar?

`make` compara las fechas de modificación.

Recompila si:

- `sensores` no existe;
- `main.cpp` es más reciente que `sensores`;
- `Alarma.cpp` es más reciente;
- `Sensor.cpp` es más reciente.

Si nada cambió, muestra:

```
make: 'sensores' está actualizado.
```

La ventaja es que no se recompila innecesariamente cada vez.

---

9. ¿Fue compilación separada?

El `Makefile` automatiza correctamente la construcción, pero compila todos los `.cpp` juntos:

```
g++ -o sensores main.cpp Alarma.cpp Sensor.cpp
```

Por eso todavía no realiza compilación separada en archivos `.o`.

Una compilación verdaderamente separada sería conceptualmente:

```
main.cpp   → main.o
Sensor.cpp → Sensor.o
Alarma.cpp → Alarma.o

main.o + Sensor.o + Alarma.o → sensores
```

El `Makefile` presentado era suficiente para introducir `make`, pero tiene esta limitación:

> Si cambia solamente `Sensor.cpp`, se vuelven a compilar también `main.cpp` y `Alarma.cpp`.

Más adelante se pueden agregar reglas independientes para cada archivo objeto.

---

10. ¿Qué concepto dejó la actividad 2?

Lo importante no era memorizar el `Makefile`, sino comprender:

- un programa puede estar formado por varios módulos;
- todos los módulos deben compilarse y enlazarse;

- `make` automatiza ese procedimiento;
- las dependencias le indican cuándo recompilar;
- `clean` permite eliminar archivos generados;
- el código fuente y el ejecutable son elementos diferentes.

Actividades 3 y 4 — Sistema de móviles

Ambas actividades utilizaron la misma consigna:

> Modelar e implementar un sistema que represente uno o más móviles capaces de desplazarse en una trayectoria bidimensional.

La actividad 3 se ocupaba principalmente del **modelo y la estructura**.

La actividad 4 completaba la **implementación y las pruebas**.

Análisis previo de la consigna

Antes de abrir Umbrello o escribir C++, había que determinar qué necesitaba representar el sistema.

11. ¿Qué información necesita conocer un móvil?

Para la solución establecemos estos atributos:

Atributo	Significado
`identificacion`	Permite diferenciar un móvil de otro
`enMovimiento`	Indica si está detenido o moviéndose
`distanciaRecorrida`	Acumula la longitud total recorrida
`posX`	Posición actual en el eje X
`posY`	Posición actual en el eje Y
`velX`	Componente de velocidad en X
`velY`	Componente de velocidad en Y

---

12. ¿Qué operaciones debía ofrecer?

A partir de la consigna se identificaron:

```
Movil();
Movil(std::string id, float x, float y);
```

```
void iniciarMovimiento();
void detener();
void setVelocidad(float vx, float vy);
bool consultarEstado();
void simular(float tiempo);
void mostrarInformacion();
```

Cada método responde a un requerimiento del tp:

Requerimiento	Método
Crear con valores predeterminados	`Movil()`
Crear indicando valores iniciales	`Movil(id, x, y)`
Consultar si se mueve	`consultarEstado()`
Comenzar a moverse	`iniciarMovimiento()`
Detenerse	`detener()`
Cambiar velocidad	`setVelocidad(vx, vy)`
Simular un intervalo	`simular(tiempo)`
Mostrar el estado	`mostrarInformacion()`

La actualización de posición y distancia no se hizo desde `main()`. Se dejó dentro de `simular()`, porque el propio móvil debe ser responsable de mantener consistente su estado.

Actividad 3 — Modelado en Umbrello

13. ¿Qué pedía concretamente?

La actividad solicitaba:

1. Instalar Umbrello.
2. Crear un proyecto UML.
3. Agregar un diagrama de clases.
4. Incorporar una nota con apellido, nombre y legajo.
5. Modelar la clase según los requerimientos.
6. Indicar atributos, visibilidad, tipos y valores predeterminados.
7. Indicar métodos, parámetros y tipos de retorno.
8. Generar código C++.
9. Revisar los archivos generados.
10. Incluir el diagrama y el código en el informe.

El propósito era diseñar este orden:

```
Requerimientos → modelo UML → código C++
```

---

14. Cómo quedó el diagrama

El diagrama de `Movil` tenía aproximadamente esta estructura:

```
UML
```

```
Movil
```

```
- identificacion : string
- enMovimiento : bool = false
- distanciaRecorrida : float = 0.0 //valor inicial
- posX : float = 0.0
- posY : float = 0.0
- velX : float = 0.0
- velY : float = 0.0
```

```
+ Movil()
+ Movil(id : string, x : float, y : float)
+ detener()
+ iniciarMovimiento()
+ mostrarInformacion()
+ setVelocidad(vx : float, vy : float)
+ consultarEstado() : bool
+ simular(tiempo : float)
```

En UML:

- `-` significa privado;
- `+` significa público;
- `nombre : tipo` representa un atributo;
- `metodo(parametro : tipo) : retorno` representa una operación;

---

15. ¿Por qué los atributos eran privados?

Los atributos se declararon privados para que no pudieran modificarse arbitrariamente desde `main()`. se debe usar la interfaz pública:

```
movil.setVelocidad(2.0, 0.0);
movil.iniciarMovimiento();
movil.simular(5.0);
```

Así, la propia clase controla cómo cambia su estado. Este es **el encapsulamiento** trabajado en la clase.

```
Datos internos privados
          ↑
Métodos públicos controlados
```

---

16. Generación de código con Umbrello

Umbrello transformó el diagrama en dos módulos:

```
Movil.h
Movil.cpp
```

`Movil.h`

Contiene la declaración de la clase:

- qué atributos posee;
- qué operaciones ofrece;
- qué elementos son públicos;
- qué elementos son privados.

`Movil.cpp`

Debe contener la implementación de los métodos:

---

17. Correcciones realizadas al código generado

Inclusión de `string`

Se reemplazó una referencia genérica por:

```
#include <string>
```

Esto es necesario porque `std::string` pertenece a la biblioteca estándar.

Uso de `std::string`

Se cambió el tipo genérico por:

```
std::string identificacion;
```

Declaraciones en el `.h`

Una declaración debe terminar con punto y coma:

```
void detener();
```

No con un cuerpo vacío:

```
void detener()
{
}
```

Si se deja un cuerpo en el `.h`, el método ya queda definido allí. Luego, al definir nuevamente:

```
void Movil::detener()
{
    enMovimiento = false;
}
```

podría producirse una redefinición.

Por eso:

```
Movil.h   → declaración
Movil.cpp → implementación
```

---

18. Otros elementos generados

Destructor

Umbrello generó:

```
virtual ~Movil();
```

Y en el `.cpp`:

```
Movil::~Movil()
{
}
```

El destructor se ejecuta cuando se destruye el objeto.

En este caso está vacío porque `Movil` no reserva memoria dinámica ni posee recursos que deban liberarse. El `virtual` tampoco era indispensable todavía, ya que no se utilizó herencia, pero **permite que el destructor se comporte correctamente si en el futuro `Movil` se usa como clase base**.

Métodos de acceso privados

Umbrello también generó varios getters y setters internos, por ejemplo:

```
void setPosX(float value);
float getPosX();
```

En tu versión quedaron privados. Por eso no forman parte de la interfaz que utiliza `main()`.

Actividad 4 — Implementación y pruebas

19. ¿Qué pedía?

La actividad solicitaba:

1. Instalar y configurar VS Code.
2. Instalar la extensión C/C++.
3. Abrir el código generado por Umbrello.
4. Completar la implementación.
5. Mantener correspondencia entre UML y código.
6. Crear `main.cpp`.
7. Crear un `Makefile`.
8. Compilar mediante `make`.
9. Ejecutar el programa.
10. Comprobar todos los requerimientos.
11. Corregir los errores.
12. Entregar proyecto y capturas.

El resultado ya no debía ser solamente un modelo. Debía convertirse en un programa funcional.

Explicación de `Movil.h`

20. Interfaz pública

La parte **pública** quedó así:

```
Movil();
virtual ~Movil();
Movil(std::string id, float x, float y);

void detener();
void iniciarMovimiento();
void mostrarInformacion();
void setVelocidad(float vx, float vy);
bool consultarEstado();
void simular(float tiempo);
```

Esto representa todo lo que el resto del programa puede pedirle a un objeto `Movil`.

---

21. Representación privada

Los atributos privados forman el estado interno:

```
std::string identificacion;
bool enMovimiento;
float distanciaRecorrida;
float posX;
float posY;
float velX;
float velY;
```

Explicación de `Movil.cpp`

22. Constructor predeterminado

```
Movil::Movil()
{
    initAttributes();
}
```

Este constructor permite:

```
Movil movilA;
```

`initAttributes()` inicia todo en 0/apagado:

Por eso `movilA` comienza:

```
sin identificación
detenido
posición (0,0)
velocidad (0,0)
distancia 0
```

---

23. Constructor **con parámetros**

```
Movil::Movil(std::string id, float x, float y)
{
    initAttributes();
    identificacion = id;
    posX = x;
```

```
    posY = y;
}
```

Permite:

```
Movil movilB("Movil_B", 10.0, 5.0);
```

Primero se inicializan todos los atributos con valores seguros. Después se reemplazan:

```
identificacion → Movil_B
posX           → 10
posY           → 5
```

La velocidad sigue siendo cero, la distancia cero y el estado detenido.

---

24. Inicio y detención

```
void Movil::detener()
{
    enMovimiento = false;
}

void Movil::iniciarMovimiento()
{
    enMovimiento = true;
}
```

Estos métodos no cambian directamente la posición. Sólo modifican el estado que posteriormente consulta `simular()`.

---

25. Modificación de velocidad

```
void Movil::setVelocidad(float vx, float vy)
{
    velX = vx;
    velY = vy;
}
```

Permite cambiar las componentes:

```
movilB.setVelocidad(2.0, 2.0);
```

Esto no desplaza inmediatamente el móvil. Solamente configura cómo se moverá durante la próxima simulación.

La separación es:

```
setVelocidad()       → configura
iniciarMovimiento()  → habilita el movimiento
simular()            → hace transcurrir el tiempo
```

---

26. Consulta del estado

```
bool Movil::consultarEstado()
{
    return enMovimiento;
}
```

Devuelve:

```
true  → en movimiento
false → detenido
```

No modifica el objeto; sólo consulta su estado.

Conceptualmente podría declararse:

```
bool consultarEstado() const;
```

El `const` indicaría formalmente que el método no cambia ningún atributo. No era indispensable para cumplir el TP, pero sería una mejora de diseño.

---

27. Método `simular()`

Este es el núcleo lógico del programa:

```
void Movil::simular(float tiempo)
{
    if (enMovimiento)
    {
        float deltaX = velX * tiempo;
        float deltaY = velY * tiempo;

        posX = posX + deltaX;
        posY = posY + deltaY;

        distanciaRecorrida =
            distanciaRecorrida + std::sqrt(deltaX * deltaX + deltaY * deltaY);
    }
}
```

Condición de movimiento

```
if (enMovimiento)
```

Cumple el requerimiento:

> Un móvil detenido no deberá modificar su posición cuando transcurra el tiempo.

Si está detenido, el método termina sin cambiar nada.

Desplazamiento en X

$$\Delta x = v_x \cdot \Delta t$$

En código:

```
float deltaX = velX * tiempo;
```

Desplazamiento en Y

$$\Delta y = v_y \cdot \Delta t$$

En código:

```
float deltaY = velY * tiempo;
```

Nueva posición

$$x_{nueva} = x_{anterior} + \Delta x$$

$$y_{nueva} = y_{anterior} + \Delta y$$

En código:

```
posX = posX + deltaX;
posY = posY + deltaY;
```

Distancia recorrida en el tramo

No podemos sumar directamente `deltaX + deltaY`. La distancia del desplazamiento bidimensional es la magnitud del vector:

$$\Delta s = \sqrt{(\Delta x)^2 + (\Delta y)^2}$$

En código:

```
std::sqrt(deltaX * deltaX + deltaY * deltaY)
```

Finalmente se acumula:

$$s_{total\ nuevo} = s_{total\ anterior} + \Delta s$$

Por eso se escribe:

```
distanciaRecorrida =
    distanciaRecorrida + std::sqrt(...);
```

Esto permite realizar varios tramos sin perder la distancia anterior.

---

28. Mostrar información

```
void Movil::mostrarInformacion()
```

muestra:

- identificación;
- estado;
- posición;
- velocidad;
- distancia total.

Si la identificación está vacía:

```
if (identificacion == "")
```

se muestra:

```
(Sin Identificacion)
```

Para el estado:

```
if (enMovimiento)
```

se elige entre:

```
En movimiento
Detenido
```

Este método centraliza la presentación del estado. `main()` no necesita consultar y formatear cada atributo por separado.

Explicación de `main.cpp`

29. Creación de dos objetos

```
Movil movilA;
Movil movilB("Movil_B", 10.0, 5.0);
```

Esto demuestra las dos formas de inicialización exigidas:

- constructor predeterminado;
- constructor con parámetros.

Estado inicial:

Móvil	Posición	Velocidad	Estado	Distancia
movilA	(0,0)	(0,0)	Detenido	0
movilB	(10,5)	(0,0)	Detenido	0

---

30. Primera simulación

Se configura:

```
movilA.setVelocidad(2.0, 0.0);
movilA.iniciarMovimiento();

movilB.setVelocidad(0.0, 3.0);
movilB.iniciarMovimiento();
```

Luego:

```
movilA.simular(5.0);
movilB.simular(5.0);
```

Móvil A

$$\Delta x = 2 \cdot 5 = 10$$

$$\Delta y = 0 \cdot 5 = 0$$

Posición:

$$(0, 0) + (10, 0) = (10, 0)$$

Distancia:

$$\sqrt{10^2 + 0^2} = 10$$

Resultado:

```
posición = (10,0)
distancia = 10
```

Móvil B

$$\Delta x = 0 \cdot 5 = 0$$

$$\Delta y = 3 \cdot 5 = 15$$

Posición:

$$(10, 5) + (0, 15) = (10, 20)$$

Distancia:

$$\sqrt{0^2 + 15^2} = 15$$

Resultado:

```
posición = (10,20)
distancia = 15
```

---

31. Segunda simulación

Se detiene el móvil A:

```
movilA.detener();
```

Y se cambia la velocidad de B:

```
movilB.setVelocidad(2.0, 2.0);
```

Después:

```
movilA.simular(3.0);
movilB.simular(3.0);
```

Móvil A

Como está detenido, no cambia:

```
posición = (10,0)
distancia = 10
```

Móvil B

$$\Delta x = 2 \cdot 3 = 6$$

$$\Delta y = 2 \cdot 3 = 6$$

Nueva posición:

$$(10, 20) + (6, 6) = (16, 26)$$

Distancia del segundo tramo:

$$\Delta s = \sqrt{6^2 + 6^2}$$

$$\Delta s = \sqrt{72} \approx 8.4853$$

Distancia acumulada:

$$s_{total} = 15 + 8.4853 = 23.4853$$

Resultado final:

```
posición = (16,26)
distancia total = 23.4853
```

Explicación del `Makefile`

El archivo fue:

```
TARGET = programa_movil
SRC = main.cpp Movil.cpp

$(TARGET): $(SRC)
    g++ -g $(SRC) -o $(TARGET)

run: $(TARGET)
    ./$(TARGET)

clean:
    rm -f $(TARGET)
```

32. Variables

```
TARGET = programa_movil
SRC = main.cpp Movil.cpp
```

---

33. Compilación

```
$(TARGET): $(SRC)
    g++ -g $(SRC) -o $(TARGET)
```

Después de reemplazar las variables equivale a:

```
programa_movil: main.cpp Movil.cpp
    g++ -g main.cpp Movil.cpp -o programa_movil
```

La opción:

```
-g
```

agrega información necesaria para depurar con `gdb`.

---

34. Ejecución

```
run: $(TARGET)
    ./$(TARGET)
```

`run` depende del ejecutable. Por eso:

```
make run
```

hace dos cosas:

1. comprueba si debe compilarse `programa_movil`;
2. lo ejecuta.

Si el programa ya está actualizado, sólo lo ejecuta.

---

35. Limpieza

```
clean:
    rm -f $(TARGET)
```

Elimina:

```
programa_movil
```

En la entrega también aparece `Movil.o`, pero este `Makefile` no lo genera ni lo elimina. Probablemente se creó mediante otra compilación o configuración de VS Code.

Una versión posterior con compilación separada debería eliminar también:

```
*.o
```

Configuración de VS Code

En tu entrega aparecen dos archivos adicionales.

`tasks.json`

Configura la tarea de construcción para ejecutar:

```
make
```

Cuando VS Code necesita compilar, no llama directamente a `g++`: delega la construcción al `Makefile`.

`launch.json`

Configura el depurador para ejecutar:

```
programa_movil
```

mediante:

```
gdb
```

También indica que antes de depurar debe ejecutarse la tarea de compilación.

La relación es:

```
Iniciar depuración
        ↓
VS Code ejecuta make
        ↓
Se genera programa_movil con -g
        ↓
gdb ejecuta programa_movil
```

Qué se aprendió realmente con el TP1

Permitió practicar el proceso completo:

1. **Automatización:** utilizar `make` para construir un programa.
2. **Análisis:** convertir una descripción textual en información y operaciones.
3. **Abstracción:** representar un móvil solamente con las características relevantes.
4. **Encapsulamiento:** mantener posición, velocidad y distancia como datos privados.
5. **Modelado:** expresar la clase mediante UML antes de programarla.
6. **Generación de código:** transformar el modelo en archivos `.h` y `.cpp`.
7. **Implementación:** completar la lógica que Umbrello no puede deducir.
8. **Instanciación:** crear dos objetos independientes de la misma clase.
9. **Mensajes:** controlar cada objeto mediante llamadas a métodos.
10. **Pruebas:** verificar constructores, estados, movimientos y distancias.

La conexión más importante entre las actividades es:

> En la actividad 3 decidiste qué es un `Movil`; en la actividad 4 programaste cómo se comporta cada objeto `Movil`; y con lo aprendido en la actividad 2 automatizaste la construcción de todo el programa.

Antes de pasar a la clase 3, deberías poder explicar estas cuatro cuestiones:

- por qué `posX`, `posY` y `distanciaRecorrida` son privadas;
- qué diferencia existe entre `Movil` y `movilA`;
- por qué `simular()` calcula la distancia internamente y no desde `main()`;
- qué relación existe entre `Movil.h`, `Movil.cpp`, `main.cpp` y el `Makefile`.

Clase 3 — UML, relaciones entre clases y serialización

Ahora la clase 3 agrega tres ideas centrales:

1. UML no sirve solamente para dibujar clases aisladas: permite **representar diferentes perspectivas** del sistema.
2. Los objetos normalmente colaboran, por lo que debemos **modelar relaciones** entre sus clases.
3. Cuando el programa termina, los objetos desaparecen de memoria; para conservarlos necesitamos serialización y persistencia.

Parte A — Introducción a UML

1. Vínculo y relación

Vínculo

Un vínculo es una conexión concreta entre **dos objetos** particulares. Por ejemplo:

```
controladorTanque → sensorUltrasonico1
```

Estos son dos objetos específicos que se comunican durante la ejecución. Otro ejemplo:

```
movilA → trayectoria1
movilB → trayectoria2
```

Los vínculos existen entre instancias.

Relación

Una relación es la abstracción que describe un **conjunto de vínculos entre objetos de determinadas clases**.

```
Controlador se comunica con Sensor
Movil recorre Trayectoria
```

La distinción es:

Nivel	Ejemplo
Clase	`Controlador` se relaciona con `Sensor`
Objeto	`controladorTanque` se vincula con `sensorUltrasonico1`

---

2. ¿Qué es UML?

UML significa **Unified Modeling Language**, lenguaje unificado de modelado. Es un lenguaje gráfico que permite:

- especificar un sistema;
- visualizarlo;
- diseñarlo;
- representar su estructura y comportamiento.

La palabra “lenguaje” es importante: UML posee símbolos y reglas para combinarlos. No es simplemente un conjunto de dibujos libres.

---

3. Qué no es UML

UML **no es**: *un lenguaje de programación; un proceso obligatorio de desarrollo; una teoría completa; una herramienta específica como Umbrello*.

```
UML      → lenguaje y notación
Umbrello → herramienta que permite usar esa notación
```

Tampoco existe una única manera obligatoria de desarrollar un programa por utilizar UML. Se puede combinar UML con procesos iterativos, ágiles, tradicionales, etc.

---

4. Modelo y realidad

Un modelo es:

> **Una abstracción de un sistema del mundo real construida con un propósito específico.**

Representa **sólo** lo necesario para su propósito. Lo mismo ocurre con UML.

---

5. Diferentes modelos del mismo sistema

Un sistema puede observarse desde distintas perspectivas.

Modelo estructural

Representa qué elementos existen y cómo se relacionan. Ejemplos:

- diagrama de clases;

- diagrama de objetos;
- diagrama de componentes.

Modelo de comportamiento

Representa qué ocurre durante la ejecución. Ejemplos:

- diagrama de actividades;
- diagrama de estados;
- diagrama de secuencia.

Modelo arquitectónico

Muestra la organización general del software y sus componentes. Ningún diagrama describe todo el sistema. Cada uno responde preguntas diferentes. Por ejemplo, en el proyecto del móvil:

- el diagrama de **clases** responde qué atributos y métodos tiene `Movil`;
- un diagrama de **estados** podría mostrar `Detenido → En movimiento`;
- un diagrama de **secuencia** podría mostrar cómo `main` llama a `simular()`;
- un diagrama de **actividad** podría mostrar las decisiones durante la simulación.

---

6. Vistas y diagramas

Una **vista** representa un aspecto del sistema y puede estar formada por varios diagramas. La presentación menciona el modelo 4+1, que organiza el sistema desde distintas perspectivas:

- vista lógica o de diseño;
- vista de implementación;
- vista de procesos o interacción;
- vista de despliegue;
- vista de casos de uso, que conecta a las anteriores.

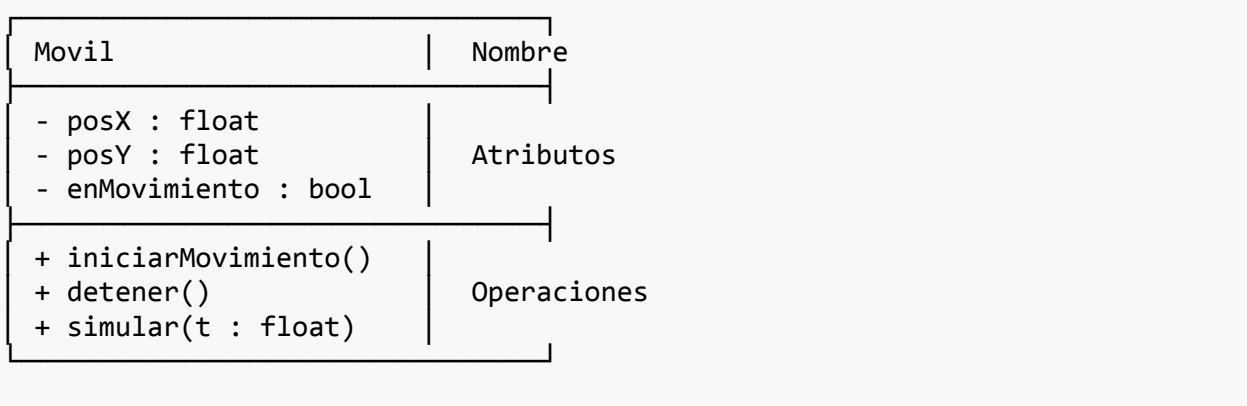
Para esta materia, los diagramas más importantes inicialmente son:

- **diagrama de clases**;
- diagrama de objetos;
- diagrama de secuencia;
- diagrama de actividad.

Parte B — Diagrama de clases

7. Cómo se representa una clase

Una clase puede mostrarse con tres compartimentos:



---

8. Visibilidad

UML utiliza símbolos para indicar visibilidad:

Símbolo	Significado	C++
+	Público	`public`
-	Privado	`private`
#	Protegido	`protected`
~	Paquete	Sin equivalencia directa simple en C++

---

9. Atributos y operaciones

Un **atributo** se representa así:

```
visibilidad nombre : tipo = valorInicial
```

Ejemplo:

```
- enMovimiento : bool = false
```

Una **operación**:

```
visibilidad nombre(parámetros) : retorno
```

Ejemplo:

```
+ consultarEstado() : bool
+ setVelocidad(vx : float, vy : float) : void
```

La notación UML no es C++. En C++:

```
float vx
```

El orden está invertido.

Parte C — Relaciones entre clases

11. Dos preguntas fundamentales: Primer Filtro

“Tiene” o “es parte de”

```
Una cinta tiene una banda.
Una rueda es parte de un automóvil.
Una computadora tiene una pantalla.
```

Aquí aparecen asociación, agregación y composición.

“Es un tipo de”

```
Un motor paso a paso es un tipo de motor.
Un Ford Fiesta es un tipo de automóvil.
Un sensor de temperatura es un tipo de sensor.
```

Aquí aparece herencia.

Una regla útil:

```
TIENE UN → asociación/agregación/composición
ES UN    → herencia
```

No siempre alcanza para decidir el tipo exacto, pero es un buen **primer filtro**.

Asociación y dependencia

12. Dependencia o asociación de uso

Una dependencia significa que una **clase necesita temporalmente a otra** para realizar alguna tarea. Se representa normalmente con una flecha discontinua:

```
Impresora - - - -> Documento
```

En C++ puede aparecer cuando una clase recibe un objeto como parámetro, pero no lo conserva:

```cpp
class Impresora
{
public:
    void imprimir(const Documento& documento);
};
```

`Impresora` necesita conocer `Documento` para ejecutar `imprimir()`, pero no necesariamente posee un documento como parte permanente de su estado. La dependencia suele ser una **relación más débil y temporal.**

---

13. Asociación

La asociación representa una conexión estructural entre clases. Ejemplo:

```
Persona conduce Taxi
```

Una `Persona` concreta puede estar vinculada con un `Taxi` concreto. En C++ puede implementarse mediante un atributo que haga referencia al otro objeto:

```cpp
class Persona
{
private:
    Taxi* taxi;
};
```

La asociación no implica necesariamente que `Persona` sea propietaria del taxi. Sólo indica que necesita mantener una conexión. La diferencia simplificada es:

Dependencia	Asociación
Uso temporal	Vínculo mantenido
Suele aparecer como parámetro o variable local	Suele aparecer como atributo
Relación débil	Relación estructural

---

14. Asociación bidireccional

Una asociación es bidireccional cuando ambos objetos pueden acceder al otro.

```
Persona ↔ Taxi
```

Podría implementarse como:

```cpp
class Persona
{
private:
    Taxi* taxi;
};
```

```
class Taxi
{
private:
    Persona* conductor;
};
```

Esto permite navegar en ambos sentidos:

```
persona → taxi
taxi → persona
```

Pero también aumenta el acoplamiento: ambas clases deben mantener consistente la relación. Por ejemplo, si una persona cambia de taxi, debemos actualizar:

- el taxi almacenado por `Persona`;
- el conductor almacenado por el taxi anterior;
- el conductor almacenado por el taxi nuevo.

Por eso **no conviene crear relaciones bidireccionales automáticamente**. Se usan cuando realmente es necesario navegar en ambos sentidos.

---

15. Asociación Unidireccional

Si sólo `Persona` necesita conocer al `Taxi`, podemos tener:

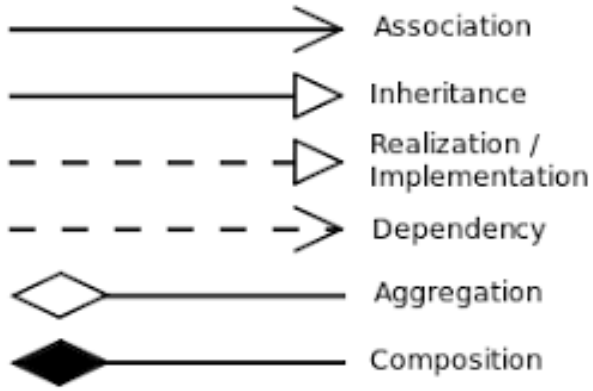
```
Persona → Taxi
```

En C++:

```
class Persona
{
private:
    Taxi* taxi;
};
```

`Taxi` no almacena ninguna referencia hacia `Persona`. La flecha de navegabilidad indica quién conoce a quién. Podemos leerla así:

> Desde un objeto `Persona` puedo llegar al `Taxi`, pero desde el taxi no necesariamente puedo llegar a la persona.



16. Creación de una Clase de asociación

A veces la relación contiene información propia. Supongamos:

```
Persona conduce Taxi
```

Si sólo necesitamos saber qué persona conduce qué taxi, alcanza con una asociación.

Pero si también necesitamos:

- fecha de inicio;
- turno;
- horas conducidas;
- número de licencia utilizada;

esa información no pertenece exclusivamente a `Persona` ni exclusivamente a `Taxi`. Pertenece al vínculo.

Entonces podemos crear una clase:

```
Conduccion

- fechaInicio
- turno
- horas
```

La relación se transforma en una entidad con información y comportamiento propios. Este criterio será importante en sistemas con **relaciones muchos-a-muchos**.

Multiplicidad

17. Qué significa

La multiplicidad indica cuántas instancias pueden participar en cada extremo.

Multiplicidad	Significado
`1`	Exactamente una
`0..1`	Ninguna o una
`*`	Cualquier cantidad
`0..*`	Cero o más
`1..*`	Una o más
`2..6`	Entre dos y seis

Ejemplo:

```
Empresa 1 ◆──────── 1..* Departamento
```

Se interpreta:

- una empresa tiene uno o más departamentos;
- cada departamento pertenece a exactamente una empresa.

La multiplicidad escrita cerca de una clase indica cuántos objetos de esa clase pueden relacionarse con un objeto del extremo opuesto.

---

18. Multiplicidad y C++

Las multiplicidades suelen orientar la estructura de datos necesaria:

UML	Posible implementación
`1`	Objeto o referencia obligatoria
`0..1`	Puntero o `std::optional`
`0..*`	`std::vector` posiblemente vacío
`1..*`	`std::vector` con validación de no vacío
`2..6`	Colección con control de tamaño

Agregación y composición

19. Agregación

La agregación representa una relación “todo-parte” débil. En UML se utiliza un **rombo blanco** junto al todo:

```
Ordenador ◇──────── Pantalla
```

La pantalla es parte del ordenador, pero puede existir independientemente:

- se puede retirar;
- reutilizar;
- conservar aunque se destruya el ordenador.

Ejemplo:

```
class Ordenador
{
private:
    Pantalla* pantalla;
};
```

El puntero señala una pantalla que podría haber sido creada en otro lugar. El ordenador la utiliza, pero no necesariamente controla su ciclo de vida. Ejemplo conceptual:

```
Pantalla pantalla;
Ordenador ordenador(&pantalla);
```

Si se destruye `ordenador`, la pantalla puede seguir existiendo.

---

20. Composición

La composición es una relación “todo-parte” fuerte. Se representa con rombo negro junto al todo:

```
Empresa ◆———————— Departamento
```

En el modelo presentado:

- el departamento pertenece a una única empresa;
- su ciclo de vida depende de ella;
- no se mueve libremente de una composición a otra;

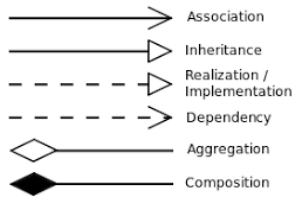
En C++ puede representarse almacenando directamente los objetos:

```
class CintaTransportadora
{
private:
    Banda banda;
    std::vector<Cilindro> cilindros;
};
```

Cuando se destruye `CintaTransportadora`, se destruyen automáticamente su `banda` y sus `cilindros`.

---

21. Diferencia esencial

Característica	Agregación	Composición	
Símbolo	Rombo blanco	Rombo negro	
Relación	Todo-parte débil	Todo-parte fuerte	
Vida de la parte	Independiente	Depende del todo	
Propiedad	Compartida o externa	Exclusiva	
Ejemplo	Equipo y pantalla	Casa y habitaciones en cierto modelo	

22. Traducción orientativa a C++

```cpp
// Asociación: conoce a otro objeto
class Controlador
{
    Sensor* sensor;
};

// Agregación: contiene objetos que pueden existir independientemente
class Laboratorio
{
    std::vector<Sensor*> sensores;
};

// Composición: posee las partes y controla su ciclo de vida
class EquipoMedicion
{
    Sensor sensorInterno;
};
```

Parte D — Herencia

23. Qué representa

La herencia modela una relación:

```
“es un tipo de”
```

Ejemplo:

```
MotorPasoAPaso es un Motor
MotorCC es un Motor
```

En UML se utiliza una línea con triángulo blanco apuntando hacia la clase base:

```
MotorPasoAPaso ————▷ Motor
MotorCC        ————▷ Motor
```

El triángulo siempre apunta hacia el concepto más general.

---

24. Clase base y clase derivada

Clase base

Contiene características comunes:

```
class Motor
{
public:
    void arrancar();
    void detener();

protected:
    float tension;
};
```

Clase derivada

Especializa la clase base:

```
class MotorPasoAPaso : public Motor
{
private:
    float anguloPaso;

public:
    void avanzar(int pasos);
};
```

`MotorPasoAPaso`:

- incluye la parte correspondiente a `Motor`;
- puede agregar atributos y métodos;
- puede redefinir comportamientos si la clase base lo permite;
- no modifica la definición de `Motor`.

Los atributos privados de la clase base forman parte del objeto derivado, aunque la clase derivada no puede accederlos directamente.

---

25. Generalización y especialización

Si observamos varias clases particulares y extraemos lo que tienen en común, realizamos una **generalización**:

```
MotorCC
MotorPasoAPaso
      ↓ extraemos lo común
Motor
```

Si partimos de una clase general y agregamos características particulares, realizamos una **especialización**:

```
Motor
  ↓ agregamos características
MotorCC
MotorPasoAPaso
```

---

26. Herencia simple y múltiple

Herencia simple

Una clase deriva de una única clase base:

```
class MotorCC : public Motor
{
};
```

Herencia múltiple

Una clase deriva de varias clases:

```
class ProfesorUniversitario :
    public Profesor,
    public Investigador
{
};
```

C++ permite herencia múltiple, pero puede generar ambigüedades.

---

27. Cuándo usar herencia

No debe utilizarse herencia solamente para reutilizar unas líneas de código. La relación debe cumplir conceptualmente:

```
ClaseDerivada ES UNA ClaseBase
```

Ejemplos razonables:

```
SensorTemperatura es un Sensor
MotorCC es un Motor
```

Ejemplo incorrecto:

```
Motor es una CintaTransportadora
```

Una cinta tiene un motor, pero no es un motor. Aquí corresponde composición o agregación.

Esta diferencia es fundamental:

```
Cinta TIENE un Motor → composición/agregación
MotorCC ES un Motor  → herencia
```

Parte E — Cómo elaborar un diagrama de clases

28. Obtener clases candidatas

La presentación propone analizar las frases nominales de la consigna. Ejemplo:

> “Una cinta transportadora es un sistema de transporte continuo formado por una banda que se mueve entre dos cilindros”.

Sustantivos candidatos:

- cinta transportadora;
- sistema de transporte;
- banda;
- cilindro.

Pero no todos los sustantivos deben transformarse automáticamente en clases.

---

29. Depurar las candidatas

Una clase resulta razonable si necesita:

- almacenar información propia;
- prestar un servicio;
- agrupar varios atributos;
- ofrecer operaciones;
- representar un requisito esencial;
- mantener una identidad independiente.

Se eliminan:

- elementos innecesarios;
- elementos que sólo serían atributos simples.

En el ejemplo, “sistema de transporte” era demasiado general y se eliminó. Quedaron:

```
CintaTransportadora
Banda
Cilindro
```

---

30. Agregar relaciones y multiplicidades

El modelo muestra que:

- la cinta posee una banda;
- la cinta posee varios cilindros.

```
CintaTransportadora 1 ——— 1 Banda
CintaTransportadora 1 ——— 0..* Cilindro
```

Luego debe decidirse si “posee” significa asociación, agregación o composición según el ciclo de vida.

---

31. Agregar atributos y métodos

Una vez obtenida la estructura general, se completa:

```
CintaTransportadora
- marca : string
- modelo : string

Banda
- largo : float
+ desplazar(longitud : float) : void

Cilindro
- radio : float
+ rotar(velocidad : float) : void
```

La secuencia recomendada es:

1. comprender la consigna;
2. obtener clases candidatas;
3. eliminar las innecesarias;
4. identificar relaciones;
5. establecer multiplicidades;
6. agregar atributos;
7. agregar responsabilidades y métodos;
8. incorporar tipos;
9. revisar el modelo;
10. modificarlo cuando aparezca nueva información.

La presentación aclara que el desarrollo rara vez es lineal. Es normal volver atrás y cambiar una clase o relación.

Parte F — Principios de diseño y GRASP

32. Responsabilidades

Diseñar objetos no consiste solamente en decidir qué atributos tienen. También debemos decidir quién es responsable de cada tarea. Una responsabilidad puede ser:

Conocer

- sus datos privados;
- los objetos relacionados;
- información que puede calcular.

Hacer

- ejecutar un cálculo;
- crear otro objeto;
- pedirle una acción a otro objeto;

---

33. Experto en información

El principio Experto indica:

> Una **responsabilidad** debería asignarse a la **clase** que **posee la información** necesaria para cumplirla.

Ejemplo. Para calcular el desplazamiento se necesitan:

- posición;
- velocidad;
- tiempo;
- distancia anterior.

`Movil` posee casi toda esa información. Por eso `simular()` pertenece a `Movil`.

No era conveniente hacer esto en `main()`:

```
movil.posX += movil.velX * tiempo;
```

Además de romper el encapsulamiento, colocaría el cálculo en una clase o función que no es experta en el estado del móvil.

---

34. Creador

El principio Creador responde: **¿Qué clase debería crear un determinado objeto?**

Una clase es buena candidata si:

- contiene al objeto;
- lo utiliza directamente;
- posee la información necesaria para inicializarlo;
- gestiona una colección de esos objetos.

En el TP1, `main()` creó los objetos porque el programa era pequeño:

```
Movil movilA;
Movil movilB;
```

En un sistema más grande podría aparecer una clase 'Simulación':

```
Simulacion
- moviles : vector<Movil>
+ crearMovil(...)
```

`Simulacion` podría ser responsable de crear móviles porque los contiene y administra.

---

36. Separación entre dominio y presentación

La presentación menciona que las clases del dominio no deberían encargarse de toda la interfaz de usuario. *En el TP1 teníamos:*

```
movil.mostrarInformacion();
```

Para un primer ejercicio es aceptable. Pero en un sistema más grande sería preferible separar:

```
Movil             → conoce su estado y movimiento
VistaMovil        → decide cómo mostrarlo
SerializadorMovil → decide cómo guardarlo
```

Así, cambiar la salida de consola por una interfaz gráfica no obliga a modificar la lógica de `Movil`.

Parte G — Persistencia y serialización

38. El problema de los objetos en memoria

Cuando ejecutamos:

```
Movil movilB("Movil_B", 10.0, 5.0);
```

el objeto existe en la memoria del proceso. Después de simular:

```
posición = (16,26)
distancia = 23.4853
```

Pero cuando termina el programa, esa memoria se libera. Si volvemos a ejecutarlo, el móvil vuelve a su estado inicial. La **persistencia busca que el estado sobreviva**:

- al final del programa;
- al cambio de computadora;
- al envío por red;

---

39. Persistencia

Un objeto es persistente cuando **su estado puede conservarse y recuperarse** más allá del lugar o momento donde fue creado.

Persistencia en el tiempo

El programa termina, pero el estado queda guardado.

```
Hoy: guardar Movil_B
Mañana: recuperar Movil_B
```

Persistencia en el espacio

La información se mueve a otra ubicación:

- otra computadora;
- un servidor;
- una base de datos.

Para lograrlo, debemos almacenar una representación del estado del objeto.

---

40. Serialización

Serializar significa **convertir un objeto o estructura en una representación lineal**:

```
Objeto → secuencia de caracteres o bytes
```

Por ejemplo, el estado:

```
identificacion = Movil_B
enMovimiento = true
posX = 16
posY = 26
velX = 2
velY = 2
distancia = 23.4853
```

puede transformarse en:

```
Movil_B,true,16,26,2,2,23.4853
```

Deserializar es realizar el proceso inverso:

```
secuencia de datos → nuevo objeto con el estado recuperado
```

Una serialización completa debe decidir:

- qué atributos guardar;
- en qué orden;
- con qué formato;
- cómo representar relaciones;
- cómo validar los datos;
- cómo reconstruir el objeto.

---

41. Serializar no es lo mismo que persistir

Podemos serializar un objeto y enviarlo por red:

```
Objeto → JSON → red
```

Aquí hubo serialización, pero no necesariamente persistencia: quizá los datos nunca se guardaron. Para persistir normalmente necesitamos:

```
Objeto
  ↓ serialización
Datos
```

```
  ↓ almacenamiento
Archivo o base de datos
```

Por eso:

> Para persistir un objeto generalmente debemos serializarlo, pero serializar no implica necesariamente persistir.

Formatos de datos

42. CSV

CSV representa datos tabulares:

```
movil,estado,posX,posY,distancia
MovilA,detenido,10,0,10
MovilB,en_movimiento,16,26,23.4853
```

Ventajas:

- simple;
- legible;
- fácil de generar;
- se abre con hojas de cálculo;
- adecuado para tablas.

Limitaciones:

- no representa bien estructuras anidadas;
- los tipos no quedan explícitos;
- requiere acordar el separador;
- debe tratar correctamente comas, comillas y saltos de línea.

Aunque CSV significa “valores separados por comas”, en configuraciones regionales también se utiliza punto y coma como separador.

---

43. XML

XML utiliza etiquetas:

```
<movil>
    <identificacion>MovilB</identificacion>
    <posX>16</posX>
    <posY>26</posY>
```

```
    <distancia>23.4853</distancia>
</movil>
```

Ventajas:

- estructura explícita;
- etiquetas con significado;
- soporta anidamiento;
- validación mediante esquemas.

Desventaja principal: suele ser **más extenso** y **verboso**.

---

44. JSON

JSON representa datos estructurados:

```
{
  "identificacion": "MovilB",
  "enMovimiento": true,
  "posicion": {
    "x": 16,
    "y": 26
  },
  "velocidad": {
    "x": 2,
    "y": 2
  },
  "distanciaRecorrida": 23.4853
}
```

Ventajas:

- legible;
- más compacto que XML;
- soporta objetos y arreglos;
- muy utilizado en redes y servicios web;
- disponible en numerosos lenguajes.

No pertenece exclusivamente a JavaScript, aunque su sintaxis provenga de allí.

45. Comparación

Formato	Mejor uso	Estructura
CSV	Tablas y registros homogéneos	Filas y columnas
XML	Documentos estructurados y muy descriptivos	Etiquetas anidadas
JSON	Intercambio de objetos y datos por red	Objetos y arreglos

Flujos en C++

46. Qué es un stream

Un stream o flujo es una abstracción para transferir datos entre un origen y un destino. Ya conocíamos:

```
std::cin
std::cout
```

También existen flujos para archivos:

Clase	Uso
`std::ifstream`	Lectura
`std::ofstream`	Escritura
`std::fstream`	Lectura y escritura

Todos permiten trabajar con una interfaz semejante:

```
flujo << dato;
flujo >> dato;
```

El destino puede cambiar, pero la idea de flujo permanece.

---

47. Escritura de un archivo

```cpp
#include <fstream>
#include <iostream>

int main()
{
    std::ofstream archivo("moviles.csv");

    if (!archivo)
    {
        std::cerr << "No se pudo abrir el archivo\n";
        return 1;
    }

    archivo << "movil,posX,posY,distancia\n";
    archivo << "MovilB,16,26,23.4853\n";

    archivo.close();

    return 0;
}
```

`ofstream` abre normalmente el archivo para escritura. Si ya existe, su contenido puede ser reemplazado. Para agregar al final:

```
std::ofstream archivo("moviles.csv", std::ios::app);
```

---

48. Lectura de un archivo

La forma segura de leer líneas es:

```
#include <fstream>
#include <iostream>
#include <string>

int main()
{
    std::ifstream archivo("moviles.csv");

    if (!archivo)
    {
        std::cerr << "No se pudo abrir el archivo\n";
        return 1;
    }

    std::string linea;

    while (std::getline(archivo, linea))
    {
        std::cout << linea << std::endl;
    }

    if (archivo.bad())
    {
        std::cerr << "Ocurrio un error de lectura\n";
    }

    return 0;
}
```

La presentación muestra una variante basada en:

```
while (!archivo.eof())
```

Pero es preferible:

```
while (std::getline(archivo, linea))
```

`eof()` se activa después de intentar leer más allá del final. Si se consulta antes, puede producirse una iteración adicional o procesarse dos veces el último dato.

49. Modos de apertura

Modo	Significado
`std::ios::in`	Lectura
`std::ios::out`	Escritura
`std::ios::app`	Agregar al final
`std::ios::binary`	Modo binario
`std::ios::trunc`	Vaciar el contenido existente

Pueden combinarse:

```
std::fstream archivo("datos.bin", std::ios::in | std::ios::out|std::ios::binary);
```

---

50. Estado del flujo

`good()`

Indica que el flujo puede continuar operando normalmente.

`eof()`

Indica que se alcanzó el final.

`fail()`

Indica que una operación falló, por ejemplo porque se esperaba un número y se encontró texto.

`bad()`

Indica un error severo de entrada/salida.

`clear()`

Limpia los indicadores de error para permitir nuevas operaciones, si corresponde. Una forma habitual y sencilla es aprovechar la conversión booleana del propio flujo:

```
if (!archivo)
{
    // La apertura u operación falló
}
```

---

51. Qué se serializa realmente en C++

C++ no serializa automáticamente un objeto completo. No alcanza con guardar directamente la memoria de cualquier clase porque puede contener:

- punteros;
- objetos dinámicos;
- tipos cuya representación depende de la arquitectura.

La dirección guardada en un puntero:

```
0x7ffc12345678
```

no sirve cuando el programa se ejecuta nuevamente. Esa dirección pertenece a una ejecución particular. Por eso, normalmente se serializan:

- identificadores;
- valores de atributos;
- cantidades;
- estados;
- relaciones mediante identificadores.

Luego se reconstruyen los objetos. Una serialización básica tampoco conserva automáticamente la identidad compartida. Si dos objetos apuntaban al mismo `Sensor`, una implementación ingenua podría reconstruir dos sensores independientes.

Conexión con los TP

La clase 3 conecta directamente con lo que ya hiciste.

TP1

El diagrama `Movil` era un diagrama de diseño:

- atributos privados;
- operaciones públicas;
- tipos;
- constructores;
- valores predeterminados.

`simular()` aplicaba el principio Experto porque `Movil` poseía posición, velocidad y distancia.

TP2

Cuando el programa escribe registros en:

```
data/prueba_arduino1.csv
```

está convirtiendo mediciones internas en una representación textual persistente.

Los datos:

```
device_id
sample_id
timestamp
position_x
position_y
```

se serializan como una fila CSV.

Cuando se vuelven a leer, se realiza el proceso inverso: interpretar el texto y reconstruir la información correspondiente.

Lo esencial de la clase 3

1. UML es un lenguaje gráfico de modelado, no un lenguaje de programación ni una metodología.
2. Un vínculo conecta objetos concretos; una relación describe conceptualmente posibles vínculos entre clases.
3. Un diagrama de clases puede incluir nombres, atributos, métodos, visibilidad, tipos, relaciones y multiplicidades.
4. Una dependencia representa uso temporal; una asociación representa un vínculo mantenido.
5. La agregación expresa una relación todo-parte débil; la composición expresa propiedad fuerte y dependencia del ciclo de vida.
6. La navegabilidad indica qué objeto conoce al otro.
7. La multiplicidad indica cuántos objetos pueden participar en cada extremo.
8. La herencia expresa “es un tipo de”; el triángulo apunta hacia la clase base.
9. El principio Experto asigna una responsabilidad a la clase que posee la información necesaria.

10. Alta cohesión significa responsabilidades relacionadas; bajo acoplamiento significa pocas dependencias innecesarias.
11. Persistir significa conservar el estado más allá de la ejecución.
12. Serializar significa convertir un objeto a una secuencia de datos; deserializar significa reconstruirlo.
13. CSV, XML y JSON son formatos posibles, cada uno adecuado para diferentes estructuras.
14. En C++, los archivos se manejan mediante streams como `ifstream` y `ofstream`.

Para verificar la comprensión antes de avanzar, las preguntas clave son:

- ¿Qué diferencia hay entre que una `Cinta` tenga un `Motor` y que `MotorCC` sea un `Motor`?
- ¿Cómo decidirías si la relación entre dos clases es agregación o composición?
- ¿Qué significa `1..*` escrito junto a una clase?
- ¿Por qué `simular()` pertenece a `Movil` según el principio Experto?
- ¿Por qué serializar un objeto y persistirlo no son exactamente lo mismo?

Clase: Excepciones en C++

Esta clase explica una forma de manejar errores que interrumpen el funcionamiento normal de una operación. Hasta ahora, una función podía comunicar un error devolviendo `false`, `-1` o algún código especial. Las excepciones ofrecen otra alternativa:

```
Se detecta el error
        ↓
se lanza una excepción
        ↓
se abandona la operación actual
        ↓
otro bloque captura y trata el error
```

1. ¿Qué es una excepción?

Una excepción es un **objeto** que **representa una situación anormal** ocurrida durante la ejecución. Por ejemplo, para calcular un área se necesitan dimensiones positivas:

```
int area(int largo, int ancho)
{
    if (largo <= 0 || ancho <= 0)
    {
        throw ErrorEnArea();
    }
```

```
    return largo * ancho;
}
```

Si los datos son correctos, se ejecuta:

```
return largo * ancho;
```

Si alguno es inválido, se ejecuta:

```
throw ErrorEnArea();
```

`throw` significa “lanzar una excepción”. Desde ese momento, la función deja de ejecutarse y C++ busca un bloque `catch` compatible.

---

2. Las tres instrucciones principales

`try`

Encierra el código que puede producir una excepción:

```
try
{
    int resultado = area(a, b);
}
```

No significa que necesariamente ocurrirá un error. Solamente indica que ese código será controlado.

`throw`

Lanza el objeto que representa el error:

```
throw ErrorEnArea();
```

La **ejecución normal se interrumpe** inmediatamente. No se ejecutan las instrucciones restantes del bloque `try`.

`catch`

Captura una excepción de un tipo determinado:

```
catch (const ErrorEnArea& error)
{
    std::cout << "Las dimensiones no son válidas\n";
}
```

Se recibe por referencia constante:

```
const ErrorEnArea& error
```

Esto evita copiar el objeto y garantiza que no será modificado.

Flujo completo

```
class ErrorEnArea
{
};

try
{
    int resultado = area(a, b);
    std::cout << resultado << "\n";
}
catch (const ErrorEnArea& error)
{
    std::cout << "No se pudo calcular el área\n";
}
```

Si `a` y `b` son positivos:

1. `area()` calcula el resultado.
2. No se ejecuta ningún `throw`.
3. Se muestra el área.
4. El `catch` se omite.

Si una dimensión es inválida:

1. `area()` ejecuta `throw`.
2. Se abandona inmediatamente la función.
3. No se muestra el resultado.
4. Se busca un `catch` para `ErrorEnArea`.
5. Se ejecuta el tratamiento del error.

La clase vacía:

```
class ErrorEnArea
{
};
```

no almacena información. Su tipo simplemente permite identificar qué error ocurrió.

---

3. La clase `std::exception`

C++ proporciona una clase general para representar excepciones:

```
#include <exception>
```

Su estructura conceptual es:

```
class exception
{
public:
    exception() noexcept;
    virtual ~exception() noexcept;
    virtual const char* what() const noexcept;
};
```

En la presentación aparece la sintaxis antigua:

```
throw()
```

En C++ moderno se utiliza principalmente:

```
noexcept
```

Ambas expresiones indican, en este caso, que esas funciones no deben lanzar otras excepciones.

Método `what()`

La función más importante es:

```
virtual const char* what() const noexcept;
```

Devuelve un mensaje que describe el motivo de la excepción.

Por ejemplo:

```
catch (const std::exception& error)
{
    std::cout << error.what() << "\n";
}
```

`what()` devuelve un `const char*`, es decir, una cadena de caracteres que no debe modificarse.

---

4. Creación de una excepción personalizada

La presentación define una clase `AreaEx` que hereda de `exception`:

```
class AreaEx : public std::exception
{
public:
    AreaEx(int motivo) : motivo(motivo)
    {
```

```
    }

      const char* what() const noexcept override; //what es una operación de
                                                  //la clase

private:
    int motivo;
};
```

La expresión:

```
: public std::exception
```

significa:

> `AreaEx` es una especificación `std::exception`.

Es una relación de herencia: `AreaEx` **es una** `std::exception`.

El atributo:

```
int motivo;
```

permite diferenciar distintos errores.

Sobrescritura de `what()`

```
const char* AreaEx::what() const noexcept
{
    switch (motivo)
    {
        case 1:
            return "Uno o más lados son negativos";

        case 2:
            return "Uno o más lados son cero";

        default:
            return "Error desconocido";
    }
}
```

Se sobrescribe el método virtual de la clase base. Por eso se utiliza:

```
override
```

De acuerdo con el valor de `motivo`, la misma clase puede devolver diferentes mensajes.

---

5. Lanzamiento desde `superficie()`

La función presentada controla dos casos:

```
float superficie(float base, float altura)
{
    if (base == 0 || altura == 0)
    {
        throw AreaEx(2);
    }

    if (base < 0 || altura < 0)
    {
        throw AreaEx(1);
    }

    return base*altura;
}
```

El comportamiento es:

Datos	Resultado
Ambos positivos	Devuelve `base * altura`
Alguno vale cero	Lanza `AreaEx(2)`
Alguno es negativo	Lanza `AreaEx(1)`

El constructor recibe el motivo:

```
throw AreaEx(2);
```

y lo guarda dentro del objeto excepción.

---

6. Captura en `main()`

```
try
{
    superficieCalculada = superficie(base, altura);
    std::cout << "Superficie: " << superficieCalculada << "\n";
}
catch (const AreaEx& error)
{
    std::cout << "Ocurrió una excepción: " << error.what() << "\n";
}
```

Si la función devuelve normalmente, se muestra la superficie.

Si lanza `AreaEx`, se salta directamente al `catch` y se muestra el mensaje correspondiente:

```
Ocurrió una excepción: Uno o más lados son negativos
```

También podría capturarse mediante la clase base:

```
catch (const std::exception& error)
{
    std::cout << error.what() << "\n";
}
```

Esto funciona porque `AreaEx` hereda de `std::exception`.

---

7. Una precisión sobre la entrada de texto

La clase contempla conceptualmente un mensaje para cuando el dato no sea numérico. Sin embargo, esta instrucción:

```
std::cin >> base;
```

normalmente no lanza automáticamente una `AreaEx` cuando se escribe texto. `cin` entra en estado de error.

Para tratarlo con la excepción personalizada habría que comprobarlo:

```
if (!(std::cin >> base))
{
    throw AreaEx(3);
}
```

De esa manera, el motivo `3` representaría realmente una entrada no numérica.

---

8. Relación con tu TP2

En tu TP2 no creaste una excepción personalizada, pero sí utilizaste las excepciones proporcionadas por C++:

```
try
{
    int device_id = std::stoi(fields[0]);
    unsigned long sample_id = std::stoul(fields[1]);
    double position_x = std::stod(fields[3]);
}
catch (const std::exception&)
{
```

```
    return false;
}
```

Las conversiones pueden lanzar, entre otras:

- `std::invalid_argument`: el texto no representa un número;
- `std::out_of_range`: el número excede el rango permitido.

Ambas derivan de `std::exception`. Por eso pueden capturarse juntas:

```
catch (const std::exception&)
```

En tu solución se combinan dos mecanismos:

- las excepciones controlan errores producidos durante las conversiones;
- el valor `false` comunica al resto del programa que el registro no pudo procesarse.

Idea fundamental

Las excepciones separan:

```
Código que detecta el problema
            ↓ throw
Código responsable de solucionarlo o informarlo
            ↓ catch
```

Lo esencial de la clase es recordar:

```
try
{
    // Operación que puede fallar
}
catch (const TipoDeExcepcion& error)
{
    // Tratamiento del error
}
```

y que la excepción se origina mediante:

```
throw TipoDeExcepcion();
```

1. ¿Qué problema debía resolver el TP 2_Req1?

La aplicación debía registrar el movimiento simulado por un Arduino.

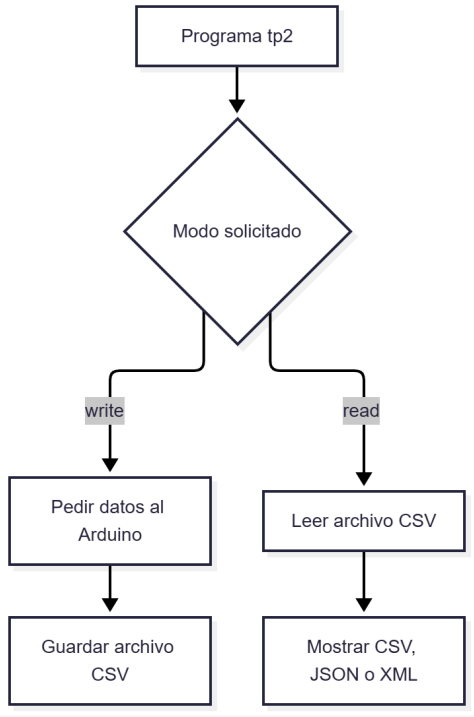
El Arduino va produciendo datos como estos:

```
2,1,1000,1.25,0.75,1.25,0.75,1.46,1
```

Cada línea describe el estado del móvil en un instante:

Campo	Valor	Significado
`device_id`	2	Dispositivo que produjo la muestra
`sample_id`	1	Número de muestra
`timestamp`	1000	Tiempo simulado, en ms
`position_x`	1.25	Posición en X
`position_y`	0.75	Posición en Y
`velocity_x`	1.25	Velocidad en X
`velocity_y`	0.75	Velocidad en Y
`total_distance`	1.46	Distancia acumulada
`status`	1	Está en movimiento

El programa debía poder hacer dos cosas:



Por eso la consigna da dos ejemplos:

```
./tp2 --mode write --file trayectoria --port /dev/ttyUSB0 --count 5
```

y:

```
./tp2 --mode read --file trayectoria --format JSON
```

En el primero se captura información. En el segundo se recupera y presenta información.

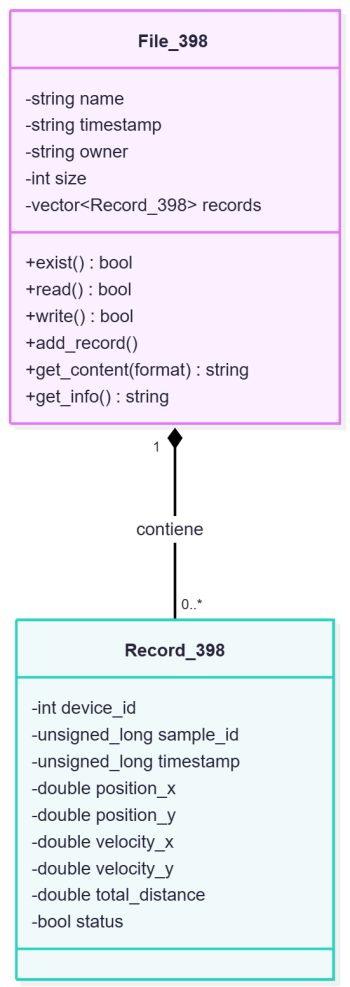
---

2. ¿Dónde aparece la orientación a objetos?

La consigna pidió reconocer las entidades fundamentales. En este problema se identificaron dos:

- `Record_398`: representa una muestra del móvil.
- `File_398`: representa un archivo y administra las muestras que contiene.

La relación es:



Esto se lee:

> Un objeto `File_398` contiene cero o muchos objetos `Record_398`.

Una analogía útil

Pensá en una planilla:

- cada fila es un `Record_398`;
- la planilla completa es un `File_398`;
- `records` es la colección de filas;
- el archivo CSV es la representación de esa planilla en el disco.

La distinción más importante es:

> `Record_398` no es el móvil: es una fotografía del móvil en un instante.

Por ejemplo:

```
Móvil en t = 1000 ms → Record 1
Móvil en t = 2000 ms → Record 2
Móvil en t = 3000 ms → Record 3
```

Cada registro es un objeto independiente.

---

3. ¿Qué se pidió y cómo se resolvió?

Requisito	Solución implementada	Motivo
Capturar datos de Arduino	Comunicación mediante `/dev/ttyUSB0`	Es el puerto serie disponible en Ubuntu
Solicitar una cantidad de registros	Opción `--count`	El operador decide cuántas muestras guardar
Guardar los datos	`File_398::write()`	Centraliza la responsabilidad del archivo
Almacenar en CSV	Archivo `data/nombre.csv`	Es el formato producido por Arduino
Leer un archivo existente	`File_398::read()`	Reconstruye los objetos guardados
Mostrar CSV, JSON o XML	`get_content(format)`	Separa el formato presentado del almacenado
Usar una CLI Linux	`getopt_long()`	Permite opciones como `--mode` y `--file`
Fijar directorio y extensión	`data/` y `.csv` en `get_path()`	El operador da solamente el nombre
Detectar errores	Métodos `bool` y mensajes en `main()`	Evita continuar con datos inválidos
Modelar con UML	Archivo `tp.xmi`	Define clases, atributos, métodos y relación
Automatizar compilación	`Makefile`	Compila todos los módulos correctamente

Una **CLI** (*Command Line Interface* o **Interfaz de Línea de Comandos**) es una forma de interactuar con una computadora usando únicamente **texto**. En lugar de hacer clic en iconos, ventanas o botones con el ratón (que es una GUI o interfaz gráfica), escribes órdenes con el teclado y el sistema te devuelve respuestas en texto.

4. Análisis del firmware `movil_csv.ino`

El Arduino es el productor de los datos. La aplicación C++ es el consumidor.

4.1 Variables que representan el estado

```
const int DEVICE_ID = 2;

unsigned long sample_id = 0;
unsigned long simulated_time = 0;

float position_x = 0.0;
float position_y = 0.0;
float velocity_x = 1.25;
float velocity_y = 0.75;
float total_distance = 0.0;
bool moving = true;
```

Estas variables permanecen en la memoria del Arduino mientras la simulación se ejecuta.

4.2 Configuración del puerto

```
void setup()

{
    Serial.begin(19200);
    Serial.println("# Simulador de movil listo");
    Serial.println("# c: registro, h: encabezado, ?: ayuda");
}
```

`Serial.begin(19200)` indica que la comunicación utiliza 19200 bits por segundo.

Arduino imprime dos mensajes informativos al iniciarse. Comienzan con # para poder distinguirlos de los datos CSV.

4.3 Recepción de comandos

```
void loop()

{
    if (Serial.available() > 0) {
        const char option = Serial.read();

        if (option == 'c') {
            advance_mobile();
            write_record();
        } else if (option == 'h') {
            write_header();
        } else if (option == '?') {
            Serial.println("# c: registro, h: encabezado, ?: ayuda");
        }
    }
}
```

Arduino no genera datos continuamente. Espera una petición de ‘c’.

4.4 Movimiento simulado

```
void advance_mobile() {
    const float delta_time = 1.0;

    ++sample_id;
    simulated_time += 1000;
```

Cada petición representa un segundo de simulación:

- `sample_id` aumenta una unidad;
- `simulated_time` aumenta 1000 ms.

Luego:

```
if (sample_id % 5 == 0)

{
    moving = !moving;
}
```

El operador `%` calcula el resto de una división. Cuando `sample_id` es divisible por 5, se cambia el estado.

Por ejemplo:

```
1 % 5 = 1 → no cambia
4 % 5 = 4 → no cambia
5 % 5 = 0 → cambia
```

Como el móvil inicialmente está moviéndose, en la muestra 5 pasa a estar detenido.

Después:

```
if (moving)

{
    position_x += velocity_x * delta_time;
    position_y += velocity_y * delta_time;

    total_distance += sqrt(
        velocity_x * velocity_x +
        velocity_y * velocity_y
    ) * delta_time;
}
```

4.5 Generación de la línea CSV

```
void write_record() {
    Serial.print(DEVICE_ID);
    Serial.print(',');
    Serial.print(sample_id);
    Serial.print(',');
    Serial.print(simulated_time);
```

Arduino intercala cada valor con una coma. *Continúa con los próximos valores*.

Cuando el móvil está detenido:

```
Serial.print(moving ? velocity_x : 0.0, 2);
```

Esta es una expresión condicional:

```
condicion ? valor_si_verdadero : valor_si_falso
```

Equivale a:

```
if (moving) {
    Serial.print(velocity_x, 2);
} else {
    Serial.print(0.0, 2);
}
```

Finalmente:

```
Serial.println(moving ? 1 : 0);
```

El booleano se representa en el CSV de esta forma:

- `1`: en movimiento;
- `0`: detenido.

---

5. Análisis de `Record_398`

5.1 El archivo de encabezado

En `Record_398.h` se encuentra la declaración de la clase.

```
#ifndef RECORD_398_H
#define RECORD_398_H
```

Estas líneas son guardas de inclusión. Evitan que el compilador procese dos veces la misma declaración si el archivo se incluye desde varios lugares.

Al final aparece:

```
#endif
```

5.2 Parte pública y parte privada

```
class Record_398
{
public:
    Record_398();
    virtual ~Record_398();

    Record_398(
        int device_id,
        unsigned long sample_id,
        ...
    );

    int get_device_id() const;
    unsigned long get_sample_id() const;
    ...
```

Lo público constituye la interfaz de la clase: aquello que otros objetos pueden utilizar.

Después aparece:

```
private:
    int device_id;
    unsigned long sample_id;
    ...
```

Los datos son privados. Desde `main.cpp` se debe interactuar mediante los métodos públicos.

5.3 Constructor vacío

```
Record_398::Record_398()
{
    initAttributes();
}
```

Este constructor crea un registro inicialmente vacío. Esto evita tener atributos sin inicializar, que podrían contener valores indeterminados.

5.4 Constructor con parámetros

```
Record_398::Record_398(
    int device_id,
    unsigned long sample_id,
    unsigned long timestamp,
    double position_x,
    ...
)
    : device_id(device_id),
```

```
      sample_id(sample_id),
      timestamp(timestamp),
      position_x(position_x),
      ...
{
}
```

La sección posterior a : es la lista de inicialización. Por ejemplo:

5.5 Getters

Un getter devuelve un atributo:

```
double Record_398::get_position_x() const
{
    return position_x;
}
```

Se utiliza así:

```
double x = record.get_position_x();
```

El `const` final indica que consultar la posición no modifica el registro.

No hay setters. Eso tiene sentido porque los registros representan estados históricos: una vez creada la muestra, **no debería modificarse arbitrariamente**.

---

6. Análisis de `File_398`

Esta es la clase con mayor responsabilidad.

6.1 Atributos

```
std::string name;
std::string timestamp;
std::string owner;
int size;


std::vector<Record_398> records;
```

Los primeros cuatro describen al archivo.

El último contiene su información:

```
std::vector<Record_398> records;
```

Un `vector` funciona como una colección dinámica. A diferencia de un arreglo tradicional, **puede crecer** mientras el programa está ejecutándose.

Ejemplo conceptual:

```
records[0] → muestra 1
records[1] → muestra 2
records[2] → muestra 3
```

6.2 Constructor

```
File_398::File_398(std::string name, std::string owner)
{
    initAttributes();

    this->name = name;
    this->owner = owner;
}
```

Primero se inicializan todos los atributos. Después se reemplazan el nombre y el propietario.

`this->name` significa:

> El atributo `name` del objeto actual.

El `name` sin `this->` es el parámetro recibido.

Si se ejecuta:

```
File_398 file("trayectoria", "pedro");
```

queda:

```
file.name = "trayectoria"
file.owner = "pedro"
```

6.3 Construcción de la ruta

```
std::string File_398::get_path() const
{
    return "data/" + name + ".csv";
}
```

Si el operador escribe:

```
--file trayectoria
```

el programa genera internamente:

```
data/trayectoria.csv
```

Esto cumple dos requisitos:

- el directorio está fijado por código;

- la extensión está fijada por código.

El operador no necesita escribir ni `data/` ni `.csv`.

6.4 Comprobación de existencia

```
bool File_398::exist()
{
    std::ifstream file(get_path());
    return file.is_open();
}
```

Se intenta abrir el archivo para lectura.

- si se puede abrir, devuelve `true`;
- si no se puede, devuelve `false`.

En modo lectura evita intentar recuperar un archivo inexistente. En modo escritura evita sobrescribir uno existente.

---

7. El método más importante: `parse_record()`

Su objetivo es transformar texto en un objeto.

```
bool File_398::parse_record(const std::string& line, Record_398& record)const
```

Recibe:

- `line`: línea CSV;
- `record`: objeto donde se colocará el resultado.

El segundo parámetro se pasa por **referencia** porque **el método necesita modificar el objeto original**.

7.1 Separación en campos

```
std::stringstream stream(line);
std::vector<std::string> fields;
std::string field;
```

Se crea un flujo llamado ‘stream’ que permite tratar el texto como una entrada de datos.

Después:

```
while (std::getline(stream, field, ','))
{
    fields.push_back(field);
}
```

A diferencia de `getline()` aplicado sobre un archivo, aquí el delimitador no es el salto de línea sino la coma.

Para:

```
2,1,1000,1.25
```

las iteraciones producen:

```
field = "2"
field = "1"
field = "1000"
field = "1.25"
```

Cada campo se agrega al vector.

7.2 Cantidad de campos

```
if (fields.size() != 9)
{
    return false;
}
```

Un registro debe contener exactamente nueve columnas, de otro modo, se rechaza.

7.3 Conversión de tipos

Todos los campos son inicialmente texto. Por ejemplo, `"1000"` no es todavía un número.

```
int device_id = std::stoi(fields[0]);
unsigned long sample_id = std::stoul(fields[1]);
double position_x = std::stod(fields[3]);
```

Las funciones realizan:

```
"2"     → entero 2
"1000"  → unsigned long 1000
"1.25"  → double 1.25
```

Esto es necesario porque el objeto `Record_398` almacena números, no cadenas.

7.4 Estado booleano

```
bool status;
if (fields[8] == "1")
{
    status = true;
}
else if (fields[8] == "0")
{
    status = false;
}
else
```

```
{
    return false;
}
```

Se acepta únicamente `0` o `1`. Una línea con:

```
...,detenido
```

sería rechazada.

7.5 Construcción del registro

```
record = Record_398(
    device_id,
    sample_id,
    record_timestamp,
    position_x,
    position_y,
    velocity_x,
    velocity_y,
    total_distance,
    status
);
```

En este punto ocurre la transformación fundamental:

```
Línea CSV
    ↓
Valores convertidos
    ↓
Objeto Record_398
```

El bloque completo está dentro de:

```
try

{
    ...
}
catch (const std::exception&)

{
    return false;
}
```

Si llega un campo como `"hola"` donde debería haber un número, `stoi()` o `stod()` produce una excepción. El método la captura y comunica que el registro no es válido.

---

8. Agregar un registro a la colección

```
void File_398::add_record(Record_398 record)
{
    records.push_back(record);
    size = static_cast<int>(records.size()) + 1;
}
```

`push_back()` agrega un objeto al final del vector.

Si el vector estaba así:

```
[registro1, registro2]
```

después de `push_back(registro3)` queda:

```
[registro1, registro2, registro3]
```

Luego se actualiza `size`.

¿Por qué se suma uno?

Porque `size` se definió como cantidad de líneas del archivo:

```
1 encabezado + cantidad de registros
```

Para cinco registros:

$$size = 5 + 1 = 6$$

Por eso la ejecución mostró:

```
Lineas: 6
```

El método:

```
bool File_398::add_record_from_csv(const std::string& line)
```

combina las **dos operaciones:**

1. analizar la línea;
2. agregar el objeto si es válido.

```
Record_398 record;

if (!parse_record(line, record))

{
    return false;
}
```

```
add_record(record);
return true;
```

---

9. Escritura del archivo: serialización y persistencia

```
bool File_398::write()
```

9.1 Creación del directorio

```
std::filesystem::create_directories(
    "data",
    directory_error
);
```

Si `data/` no existe, se crea. Si ya existe, puede seguir utilizándose.

9.2 Apertura del archivo

```
std::ofstream file(get_path());
```

`ofstream` significa output file stream: flujo de salida hacia un archivo.

Si el archivo no puede abrirse:

```
if (!file.is_open()) {
    return false;
}
```

9.3 Encabezado

```
file << "device_id,sample_id,timestamp,"
     << "position_x,position_y,"
     << "velocity_x,velocity_y,"
     << "total_distance,status\n";
```

El encabezado se escribe una sola vez.

9.4 Formato decimal

```
file << std::fixed << std::setprecision(2);
```

Esto hace que los números decimales tengan dos cifras:

```
1.25
0.75
5.00
```

9.5 Recorrido de los objetos

```
for (const Record_398& record : records)
```

Este `for` toma cada objeto de la colección.

- `const`: no va a modificarlo;
- `&`: lo consulta por referencia, evitando una copia;
- `record`: objeto correspondiente a la iteración actual.

Después obtiene los valores mediante getters:

```
file << record.get_device_id() << ","
     << record.get_sample_id() << ","
     << record.get_timestamp() << ","
     ...
```

Aquí sucede la serialización:

```
Objeto Record_398
    ↓
Secuencia de valores y comas
    ↓
Línea CSV
```

Y como esa línea se guarda en un disco, también sucede la persistencia.

Diferencia entre ambos conceptos

- **Serializar:** transformar el objeto en una secuencia transmisible o almacenable.
- **Persistir:** conservar esa secuencia más allá de la ejecución.

Arduino serializa cuando manda una línea por el puerto, pero no la persiste.

`File_398::write()` serializa y además persiste, porque guarda la línea en un archivo.

---

10. Lectura del archivo: deserialización

```
bool File_398::read()
```

Hace el proceso inverso.

10.1 Apertura

```
std::ifstream file(get_path());
```

`ifstream` significa input file stream: flujo de entrada desde un archivo.

10.2 Lectura del encabezado

```
std::string header;

if (!std::getline(file, header))
```

```
{
    return false;
}
```

La primera línea se almacena en `header`.

Después se compara con el encabezado esperado:

```
if (header != expected_header) {
    return false;
}
```

Esto evita interpretar como registros archivos que tengan otra estructura.

10.3 Vector temporal

```
std::vector<Record_398> loaded_records;
```

Los registros no se agregan inmediatamente a `records`. Primero se guardan en una colección temporal.

Después:

```
while (std::getline(file, line))
{
    if (line.empty()) {
        continue;
    }

    Record_398 record;

    if (!parse_record(line, record)) {
        return false;
    }

    loaded_records.push_back(record);
}
```

Por cada línea:

1. se ignoran líneas vacías;
2. se crea un registro vacío;
3. se analiza el CSV;
4. se construye el objeto;
5. se agrega al vector temporal.

Finalmente:

```
records = loaded_records;
```

Esto se hace solamente si todo fue correcto.

¿Por qué es una buena decisión?

Supongamos que hay cinco líneas y la cuarta está dañada. Si se trabajara directamente sobre `records`, quedarían tres registros cargados aunque la lectura hubiera fallado.

Usando `loaded_records`, la operación es “todo o nada”:

- si todas las líneas son correctas, se conserva la colección;
- si alguna falla, no se reemplaza el estado anterior.

---

11. Presentación como CSV, JSON o XML

Después de leer, los objetos están nuevamente en memoria.

`get_content()` recibe un formato:

```
std::string File_398::get_content(std::string format)
```

CSV

```
if (format == "CSV" || format == "csv")
```

Se escribe el encabezado y una línea por objeto.

Resultado:

```
device_id,sample_id,timestamp,position_x,...
2,1,1000,1.25,...
2,2,2000,2.50,...
```

JSON

```
else if (format == "JSON" || format == "json")
```

Genera una colección:

```
[
  {
    "device_id": 2,
    "sample_id": 1,
    "timestamp": 1000,
    "position_x": 1.25,
    "status": true
  }
]
```

La lógica:

```
if (i + 1 < records.size()) {
    output << ",";
}
```

evita colocar una coma después del último objeto, porque en JSON sería incorrecto.

XML

```
else if (format == "XML" || format == "xml")
```

Genera:

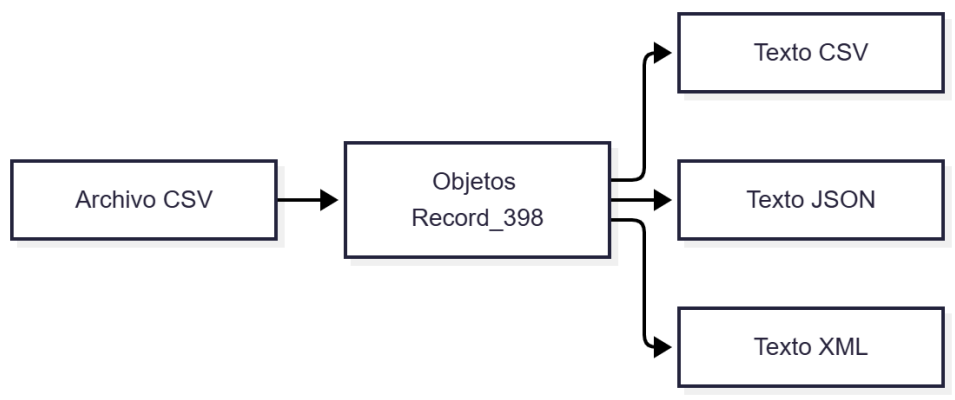
```
<records>
  <record>
    <device_id>2</device_id>
    <sample_id>1</sample_id>
    ...
  </record>
</records>
```

Punto muy importante

El archivo almacenado sigue siendo CSV.

Cuando se pide JSON o XML, el programa no convierte el archivo físicamente ni crea un archivo nuevo. Solamente cambia cómo se presentan en pantalla los objetos reconstruidos.

El recorrido es:



---

12. Información del archivo

`get_info()` utiliza `ostringstream`, que es un flujo de salida hacia una cadena.

```
std::ostringstream output;
```

No imprime inmediatamente en la consola. Primero construye un `string`.

```
output << std::left
       << std::setw(20) << "Nombre"
       << std::setw(24) << "Fecha de creacion"
       ...
```

`setw()` reserva un ancho para cada columna. Así se obtiene:

```
Nombre               Fecha de creacion       Propietario     Lineas
prueba_arduino1      2026-09-01 00:14:04     pedro           6
```

Finalmente:

```
return output.str();
```

convierte el flujo construido en un `std::string`.

---

13. Análisis de `main.cpp`

`main.cpp` es el controlador. No representa una entidad; coordina el trabajo.

Su función es:

1. interpretar lo que pidió el operador;
2. validar las opciones;
3. crear el objeto `File_398`;
4. elegir el modo;
5. comunicarse con Arduino si corresponde;
6. mostrar resultados o errores.

13.1 `argc` y `argv`

```
int main(int argc, char* argv[])
```

Si se ejecuta:

```
./tp2 --mode read --file prueba --format CSV
```

se recibe:

```
argc = 7

argv[0] = "./tp2"
argv[1] = "--mode"
argv[2] = "read"
argv[3] = "--file"
```

```
argv[4] = "prueba"
argv[5] = "--format"
argv[6] = "CSV"
```

Todos los argumentos llegan como texto, incluso `--count 5`.

Por eso `count` debe convertirse posteriormente a un entero.

13.2 Definición de opciones

```
const option long_options[] = {
    {"mode", required_argument, nullptr, 'm'},
    {"file", required_argument, nullptr, 'n'},
    {"format", required_argument, nullptr, 'f'},
    {"port", required_argument, nullptr, 'p'},
    {"count", required_argument, nullptr, 'c'},
    {"help", no_argument, nullptr, 'h'},
    {nullptr, 0, nullptr, 0}
};
```

Por ejemplo:

```
{"mode", required_argument, nullptr, 'm'}
```

indica que:

- existe la opción larga `--mode`;
- necesita un valor;
- se relaciona con la opción corta `-m`.

Estas dos órdenes son equivalentes:

```
./tp2 --mode read

./tp2 -m read
```

La cadena:

```
"m:n:f:p:c:h"
```

también describe las opciones cortas.

Los `:` indican que la opción anterior necesita un argumento:

```
m: → -m necesita un valor
n: → -n necesita un valor
f: → -f necesita un valor
p: → -p necesita un valor
c: → -c necesita un valor
h  → -h no necesita valor
```

13.3 Procesamiento

```
while ((selected_option = getopt_long(...)) != -1)
```

`getopt_long()` va procesando una opción por iteración.

Después:

```
switch (selected_option)
```

determina cuál se encontró.

Ejemplo:

```
case 'm':
    mode = optarg;
    break;
```

`optarg` contiene el valor asociado.

Para:

```
--mode write
```

queda:

```
mode = "write"
```

---

14. Normalización y validaciones

Modo

```
mode = to_lower(mode);
```

Permite escribir:

```
write
WRITE
Write
```

Todos se transforman en `write`.

Luego:

```
if (mode != "read" && mode != "write")
```

rechaza cualquier otro modo.

Cantidad

`count` llega inicialmente como texto:

```
std::string count_text;
```

`parse_count()` usa:

```
long value = std::stol(text, &processed);
```

`processed` indica cuántos caracteres pudieron convertirse.

Para:

```
"5"
```

se procesa un carácter y se acepta.

Para:

```
"5abc"
```

se puede convertir el `5`, pero no toda la cadena. Por eso se compara:

```
processed != text.size()
```

También se rechaza si:

```
value <= 0
```

Por eso `--count 0` y `--count -3` no son válidos.

Propietario

```
const char* system_user = std::getenv("USER");
```

Se consulta la variable de entorno `USER`.

En tu Ubuntu normalmente resulta:

```
pedro
```

Después se crea:

```
File_398 file(file_name, owner);
```

Este es el objeto principal del programa.

---

15. Modo `read`, paso por paso

```
if (mode == "read")
```

Paso 1: exigir formato

```
if (format.empty()) {
    ...
}
```

Sin `--format`, el programa no sabe cómo presentar los registros.

Paso 2: validar formato

```
format = to_upper(format);
```

Después acepta únicamente:

```
CSV
JSON
XML
```

Paso 3: comprobar existencia

```
if (!file.exist()) {
    ...
}
```

Paso 4: cargar los objetos

```
if (!file.read()) {
    ...
}
```

Aquí se abre el CSV, se valida el encabezado y se reconstruyen los registros.

Paso 5: mostrar resultado

```
std::cout << file.get_info() << "\n";
std::cout << file.get_content(format);
```

Primero se muestra la información del archivo. Después, los registros.

---

16. Comunicación serie del modo `write`

Esta es la parte más específica de Linux.

16.1 Apertura

```
int descriptor = open(
    port.c_str(),
    O_RDWR | O_NOCTTY | O_SYNC
);
```

El descriptor es un número entero que Linux utiliza para identificar el recurso abierto.

Por ejemplo:

```
descriptor = 3
```

No significa que sea el puerto 3. Es simplemente el identificador interno asignado por el sistema operativo.

Las opciones indican:

- `O_RDWR`: lectura y escritura;
- `O_NOCTTY`: no usar el puerto como terminal de control;
- `O_SYNC`: realizar las operaciones de manera sincronizada.

16.2 Configuración

```
cfsetispeed(&configuration, B19200);
cfsetospeed(&configuration, B19200);
```

La velocidad debe coincidir con:

```
Serial.begin(19200);
```

Si Arduino estuviera en 9600 y C++ en 19200, los datos no se interpretarían correctamente.

Después se configuran:

```
8 bits
sin paridad
1 bit de parada
sin control de flujo
```

16.3 Lectura de una línea

```
ssize_t received = read(
    descriptor,
    &character,
    1
);
```

Se lee un carácter por vez.

Si llega:

```
2,1,1000\r\n
```

el ciclo recibe sucesivamente:

```
'2'
','
'1'
','
'1'
```

```
'0'
'0'
'0'
'\r'
'\n'
```

El `\r` se ignora:

```
if (character != '\r') {
    line += character;
}
```

El `\n` marca el final:

```
if (character == '\n') {
    return !line.empty();
}
```

Por lo tanto, el resultado final queda:

```
2,1,1000,...
```

sin caracteres de final de línea.

16.4 Petición de una muestra

```
const char command = 'c';

write(descriptor, &command, 1);
```

La computadora envía el carácter `c`.

Arduino ejecuta:

```
advance_mobile();
write_record();
```

La respuesta se recoge con `read_serial_line()`.

Si empieza con `#`, se considera un mensaje informativo:

```
if (!line.empty() && line.front() != '#') {
    return true;
}
```

Solamente una línea que no comience con `#` se toma como registro.

---

17. Modo `write`, paso por paso

Después de validar el puerto y `count`, el programa comprueba:

```
if (file.exist())
```

Esto evita destruir un archivo anterior.

Luego abre el puerto:

```
int serial_descriptor = open_serial_port(port);
```

Espera dos segundos porque Arduino puede reiniciarse:

```
sleep(2);
```

Y descarta mensajes pendientes:

```
tcflush(serial_descriptor, TCIOFLUSH);
```

Después comienza el ciclo:

```
for (int index = 0; index < count; index++)
```

Para `count = 5`, `index` toma:

```
0, 1, 2, 3, 4
```

Pero al mostrar el número de registro se utiliza:

```
index + 1
```

Por eso el usuario ve del 1 al 5.

En cada iteración:

```
std::string line;
```

crea una cadena para la respuesta.

```
request_record(serial_descriptor, line)
```

pide un registro.

Después:

```
file.add_record_from_csv(line)
```

lo valida y transforma en un objeto.

Cuando las cinco muestras están cargadas:

```
close(serial_descriptor);
```

se cierra el puerto.

Finalmente:

```
file.write();
```

guarda todos los objetos.

Es importante observar el orden:

```
Primero se obtienen todos los registros
Después se crea el archivo
```

Si falla el registro 4, no se escribe un archivo incompleto con solamente tres registros. En ese sentido, la captura funciona como una operación completa.

---

18. Recorrido concreto de tu prueba

Ejecutaste:

```
./tp2 --mode write --file prueba_arduino1 --port /dev/ttyUSB0 --count 5
```

El flujo real fue:

Registro 1

Arduino recibe `c`:

```
sample_id = 1
timestamp = 1000
position = (1.25, 0.75)
distance = 1.46
status = 1
```

Envía:

```
2,1,1000,1.25,0.75,1.25,0.75,1.46,1
```

C++ crea el primer `Record_398`.

Registro 2

```
2,2,2000,2.50,1.50,1.25,0.75,2.92,1
```

Se agrega el segundo objeto.

Registro 3

```
2,3,3000,3.75,2.25,1.25,0.75,4.37,1
```

Registro 4

```
2,4,4000,5.00,3.00,1.25,0.75,5.83,1
```

Registro 5

En la quinta muestra cambia `moving` a `false`:

```
2,5,5000,5.00,3.00,0.00,0.00,5.83,0
```

El vector queda conceptualmente así:

```
records = [
    Record_398(muestra 1),
    Record_398(muestra 2),
    Record_398(muestra 3),
    Record_398(muestra 4),
    Record_398(muestra 5)
]
```

Después `write()` crea:

```
data/prueba_arduino1.csv
```

con seis líneas: encabezado más cinco registros.

---

19. El `Makefile`

```
CXX = g++
CXXFLAGS = -g -Wall -Wextra -std=c++17
```

- `g++`: compilador;
- `-g`: información para depuración;
- `-Wall -Wextra`: advertencias;
- `-std=c++17`: versión de C++ requerida por `filesystem`.

```
TARGET = tp2
SRC = main.cpp File_398.cpp Record_398.cpp
```

Se define el nombre del ejecutable y los fuentes.

```
$(TARGET): $(SRC)
    $(CXX) $(CXXFLAGS) $(SRC) -o $(TARGET)
```

Se expande aproximadamente como:

```
g++ -g -Wall -Wextra -std=c++17 \
    main.cpp File_398.cpp Record_398.cpp \
    -o tp2
```

También existen:

```
make run
make debug
make clean
```

Pero `make run` ejecuta `./tp2` sin argumentos, por lo que solamente mostrará el error de opciones faltantes. Sería más útil permitir pasar argumentos mediante una variable.

20. Qué conceptos de la materia se aplicaron

Concepto	Aplicación en el TP
Abstracción	Se reconocen `File_398` y `Record_398`
Encapsulamiento	Los atributos son privados
Clase	Plantilla que define atributos y métodos
Objeto	Cada muestra concreta es un `Record_398`
Composición	`File_398` contiene los registros
Multiplicidad	Un archivo contiene `0..*` registros
Colección	`std::vector<Record_398>`
Responsabilidad	Cada clase administra sus propios datos
Serialización	Objeto → CSV, JSON o XML
Deserialización	CSV → objeto
Persistencia	Los registros se guardan en disco
Streams	`ifstream`, `ofstream`, `stringstream`
Excepciones	Detección de conversiones numéricas inválidas
Separación de capas	`main.cpp` coordina; las clases manejan el modelo
CLI	`argc`, `argv` y `getopt_long()`

21. Aspectos que están bien y aspectos mejorables

La solución central funciona correctamente. La compilé sin advertencias y probé los tres formatos.

Correcto

- comunicación real con Arduino;
- coincidencia de velocidad a 19200;
- validación de cantidad;
- rechazo de archivos inexistentes;

- prevención de sobrescritura;
- verificación del encabezado;
- nueve campos obligatorios;
- reconstrucción de objetos;
- almacenamiento en `vector`;
- escritura CSV;
- generación válida de JSON y XML;
- separación razonable entre `main` y las clases.

Mejorable

`owner`

En lectura muestra el usuario que ejecuta el programa, no necesariamente quien creó el archivo. No está almacenado en el CSV.

Fecha de creación

En lectura se utiliza `last_write_time()`, que representa la última modificación. No necesariamente la creación.

Esto explica por qué en tu ejecución la hora mostrada al leer no era exactamente la misma que al crear.

Validación numérica

`parse_record()` detecta valores completamente inválidos, pero podría controlar también cadenas como:

```
1.25abc
```

de la misma manera rigurosa que `parse_count()`.

Nombre del archivo

Debería impedir nombres con `/`, `..` o extensiones manuales para garantizar que todo quede dentro de `data/`.

`Makefile`

No incluye los `.h` como dependencias. Si cambia solamente un encabezado, puede ser necesario forzar la recompilación con:

```
make clean
make
```

Destructor virtual

```
virtual ~Record_398();
```

y:

```
virtual ~File_398();
```

no son necesarios porque no se está usando herencia. Probablemente fueron generados automáticamente por Umbrello y no causan un error, pero agregan una característica innecesaria.

---

22. Resumen para entenderlo y defenderlo

Podrías explicarlo así:

> El firmware simula el movimiento de un dispositivo y, cuando recibe el comando `c`, genera una muestra en formato CSV. La aplicación C++ se controla mediante argumentos de línea de comandos. En modo escritura abre el puerto serie, solicita la cantidad indicada de muestras, valida cada línea y la convierte en un objeto `Record_398`. Los objetos son almacenados en el vector perteneciente a `File_398`, cuya relación es de composición. Una vez completada la captura, `File_398::write()` serializa los objetos y los persiste en un archivo CSV dentro del directorio `data`. En modo lectura, `File_398::read()` realiza la operación inversa: valida el encabezado, deserializa cada fila y reconstruye la colección. Finalmente, `get_content()` presenta los registros en CSV, JSON o XML.

La secuencia fundamental que tenés que recordar es:

```
MODO WRITE
Arduino
→ línea CSV
→ parse_record()
→ objeto Record_398
→ vector de File_398
→ write()
→ archivo CSV

MODO READ
archivo CSV
→ read()
→ parse_record()
→ objetos Record_398
→ get_content()
→ CSV, JSON o XML en pantalla
```

Si comprendés esas dos cadenas, ya comprendés la lógica principal del TP. El resto —`termios`, `getopt_long`, `fstream`— son mecanismos técnicos utilizados para hacer posible cada etapa.